Университет ИТМО

# Отчёт по домашнему заданию № 3

«Технология MapReduce»

Вариант 4: «Программист, разработчик на C# и .NET»

Выполнил: студент группы J3310
Бровкин Аким Алексеевич
ИСУ 465282

Санкт-Петербург, 2026

# 1 Задача, вариант и план

Нужно было собрать небольшой аналитический пайплайн. Данные лежат в объектном хранилище, навыки и их сочетания я считаю двумя независимыми способами и сверяю результаты. Мой вариант — **«Программист, разработчик на C# и .NET»**.В подстроку `.net` попадают и ASP.NET, и VB.NET. Весь код, скрипты запуска, выводы команд и итоговые таблицы лежат в репозитории

`github.com/seichikk/hh-mapreduce`

# 2 Стенд

| | |
|---|---|
| Компьютер | MacBookPro16,1 |
| Процессор | Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz (физических ядер: 6) |
| Память | 32 ГБ, из них Docker выделено 5920 МиБ и 12 CPU |
| Система | macOS 26.6.2 |
| Хранилище | RustFS 1.0.1 в Docker, 4 папки-диска, схема EC $2+2$ |
| Вычисления | Python 3.12, DuckDB 1.5.6 с расширением httpfs, pyarrow, boto3 (в отдельном контейнере) |

**Таблица 1:** Стенд

```
==== стенд ====
модель: MacBookPro16,1
процессор: Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz, физических ядер: 6
оперативная память: 32 гб
система: macOS 26.6.2
docker engine 29.8.0, выделено docker: 12 cpu, 5920 миб памяти
python 3.12.15 | duckdb 1.5.6 | pyarrow 26.0.0 | boto3 1.43.116

==== датасет ====
файл: dataset/it_vacancies_2024.csv, 847619384 байт
считаю sha256 (секунд 10)...
sha256: 303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
это тот самый курсовой файл it_vacancies_2024.csv (sha256 совпал с эталоном курса)
cut: stdin: Illegal byte sequence
source,vacancy_id,status,snapshot_date,published_at,area,employer_id,employer_inn,employer_ogrn,employer_name,n⌋
↪  ame,role,sphere,salary_from,salary_to,currency,gross,experience,schedule,employment,key_skills,description,⌋
↪  url
hh,106654617,not_active,2024-09-03,2024-09-02,Краснодар,4105707,,1197325010211,,Python Backend
↪  Developer,"Программист, разработчик",,,,,,От 3 до 6 лет,Удаленная работа,Полная занятость,Python;
↪  PostgreSQL; Django Framework,"В Nord Clan мы занимаемся разработкой решений и алг
```

# 3 Этапы 1–2. Хранилище и данные

Хранилище я поднял так же, как в ДЗ-2: RustFS в Docker, четыре папки-диска `disks/disk1..4`, схема Erasure Coding 2+2. Отличие одно: и RustFS, и контейнер с вычислениями подключены к общей docker-сети `hhnet`. Поэтому Python и DuckDB ходят в хранилище по адресу `http://rustfs:9000` как обычные S3-клиенты.

Датасет `it_vacancies_2024.csv` весит 808.35 миб. Он загружен в бакет `hh-lake` под ключом `raw/it_vacancies_2024.csv` составной загрузкой частями по 64 МиБ. ETag такого объекта не равен MD5 файла, поэтому целостность я проверяю иначе: читаю объект обратно из S3 и считаю SHA-256. Сумма `303db922b541e4440906f5a7e8be32fb...` совпала с локальной (да).

Это единственный шаг, которому нужен локальный файл. Всем дальнейшим вычислениям папка `dataset` в контейнер вообще не пробрасывается, и данные они берут только из бакета.

```
==== этап 1. объектное хранилище rustfs: 4 папки-диска, erasure coding ====
```

```
нашёл старый контейнер rustfs (например, из дз-2), удаляю его. данные дз-2 в ~/hdfs-lab не трогаются

$ docker run -d --name rustfs --network hhnet --user 0:0 --restart unless-stopped -p 127.0.0.1:9000:9000 -p
↪  127.0.0.1:9001:9001 -v /Users/seichik/Downloads/hh-mapreduce/disks/disk1:/data/rustfs1 -v
↪  /Users/seichik/Downloads/hh-mapreduce/disks/disk2:/data/rustfs2 -v
↪  /Users/seichik/Downloads/hh-mapreduce/disks/disk3:/data/rustfs3 -v
↪  /Users/seichik/Downloads/hh-mapreduce/disks/disk4:/data/rustfs4 -v
↪  /Users/seichik/Downloads/hh-mapreduce/logs:/logs -e RUSTFS_VOLUMES=/data/rustfs{1...4} -e
↪  RUSTFS_ACCESS_KEY=rustfsadmin -e RUSTFS_SECRET_KEY=rustfsadmin -e RUSTFS_ADDRESS=0.0.0.0:9000 -e
↪  RUSTFS_CONSOLE_ENABLE=true -e RUSTFS_CONSOLE_ADDRESS=0.0.0.0:9001 -e RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true -e
↪  RUSTFS_OBS_LOG_DIRECTORY=/logs -e RUSTFS_OBS_LOGGER_LEVEL=warn rustfs/rustfs:1.0.1
79d9f50d4b52420b45d7aad2071a7ad135b921b2eb0cd5e8b56414201e74db0f
жду, пока rustfs поднимется...
rustfs отвечает: http://localhost:9000/health/live -> 200

$ docker ps --filter name=^rustfs$ --format table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}
NAMES     IMAGE                 STATUS        PORTS
rustfs    rustfs/rustfs:1.0.1   Up 1 second   127.0.0.1:9000-9001->9000-9001/tcp

==== этап 2. загрузка датасета в бакет hh-lake ====

$ python -m lab.storage upload --path /data/it_vacancies_2024.csv
создан бакет hh-lake
локальный файл: it_vacancies_2024.csv, 847619384 байт (808.35 миб)
sha256 локального файла: 303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
загружено в s3://hh-lake/raw/it_vacancies_2024.csv за 16.5 с (49.1 миб/с)
читаю объект обратно из хранилища и считаю sha256...
sha256 объекта в s3:     303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
размер объекта в s3: 847619384 байт
файл в хранилище совпадает с исходным: да

$ python -m lab.storage inspect
колонки датасета (23): source, vacancy_id, status, snapshot_date, published_at, area, employer_id, employer_inn,
↪  employer_ogrn, employer_name, name, role, sphere, salary_from, salary_to, currency, gross, experience,
↪  schedule, employment, key_skills, description, url
все нужные колонки на месте: name, role, key_skills, area, salary_from, salary_to, currency, gross
первая запись (обрезано):
  vacancy_id: '106654617'
  area: 'Краснодар'
  name: 'Python Backend Developer'
  role: 'Программист, разработчик'
  salary_from: ''
  salary_to: ''
  currency: ''
  gross: ''
  experience: 'От 3 до 6 лет'
  schedule: 'Удаленная работа'
  key_skills: 'Python; PostgreSQL; Django Framework'

$ python -m lab.storage ls --prefix raw/
содержимое s3://hh-lake/raw/
  2026-10-20 21:52:21     847619384  raw/it_vacancies_2024.csv
объектов: 1, всего 808.35 миб

$ du -sh disks/disk1 disks/disk2 disks/disk3 disks/disk4
409M    disks/disk1
410M    disks/disk2
410M    disks/disk3
408M    disks/disk4
```

```
$ docker run -d --name rustfs --network hhnet --user 0:0 --restart unless-stopped -p 127.0.0.1:9000:9000 -p 127.0.0.1:9001:9001 -v /Users/seichik/Downloads/hh-mapreduce/disks/disk1:/data/rustfs1 -v /Users/seichik/
Downloads/hh-mapreduce/disks/disk2:/data/rustfs2 -v /Users/seichik/Downloads/hh-mapreduce/disks/disk3:/data/rustfs3 -v /Users/seichik/Downloads/hh-mapreduce/disks/disk4:/data/rustfs4 -v /Users/seichik/Downloads/hh
-mapreduce/logs:/logs -e RUSTFS_VOLUMES=/data/rustfs{1...4} -e RUSTFS_ACCESS_KEY=rustfsadmin -e RUSTFS_SECRET_KEY=rustfsadmin -e RUSTFS_ADDRESS=0.0.0.0:9000 -e RUSTFS_CONSOLE_ENABLE=true -e RUSTFS_CONSOLE_ADDRESS=
0.0.0.0:9001 -e RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true -e RUSTFS_OBS_LOG_DIRECTORY=/logs -e RUSTFS_OBS_LOGGER_LEVEL=warn rustfs/rustfs:1.0.1
79d9f50d4b52420b45d7aad2071a7ad135b921b2eb0cd5e8b56414201e74db0f
жду, пока rustfs поднимется...
rustfs отвечает: http://localhost:9000/health/live -> 200

$ docker ps --filter name=^rustfs$ --format table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}
NAMES     IMAGE                 STATUS         PORTS
rustfs    rustfs/rustfs:1.0.1   Up 1 second    127.0.0.1:9000-9001->9000-9001/tcp

==== этап 2. загрузка датасета в бакет hh-lake ====

$ python -m lab.storage upload --path /data/it_vacancies_2024.csv
создан бакет hh-lake
локальный файл: it_vacancies_2024.csv, 847619384 байт (808.35 миб)
sha256 локального файла: 303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
загружено в s3://hh-lake/raw/it_vacancies_2024.csv за 16.5 с (49.1 миб/с)
читаю объект обратно из хранилища и считаю sha256...
sha256 объекта в s3:     303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
размер объекта в s3: 847619384 байт
файл в хранилище совпадает с исходным: да

$ python -m lab.storage inspect
колонки датасета (23): source, vacancy_id, status, snapshot_date, published_at, area, employer_id, employer_inn, employer_ogrn, employer_name, name, role, sphere, salary_from, salary_to, currency, gross, experienc
e, schedule, employment, key_skills, description, url
все нужные колонки на месте: name, role, key_skills, area, salary_from, salary_to, currency, gross
первая запись (обрезано):
  vacancy_id: '106654617'
  area: 'Краснодар'
  name: 'Python Backend Developer'
  role: 'Программист, разработчик'
  salary_from: ''
  salary_to: ''
  currency: ''
  gross: ''
  experience: 'От 3 до 6 лет'
  schedule: 'Удаленная работа'
  key_skills: 'Python; PostgreSQL; Django Framework'

$ python -m lab.storage ls --prefix raw/
содержимое s3://hh-lake/raw/
  2026-10-20 21:52:21     847619384  raw/it_vacancies_2024.csv
объектов: 1, всего 808.35 миб

$ du -sh disks/disk1 disks/disk2 disks/disk3 disks/disk4
409M    disks/disk1
410M    disks/disk2
410M    disks/disk3
408M    disks/disk4
```

**Рис. 1:** Запуск RustFS, загрузка датасета и сверка SHA-256

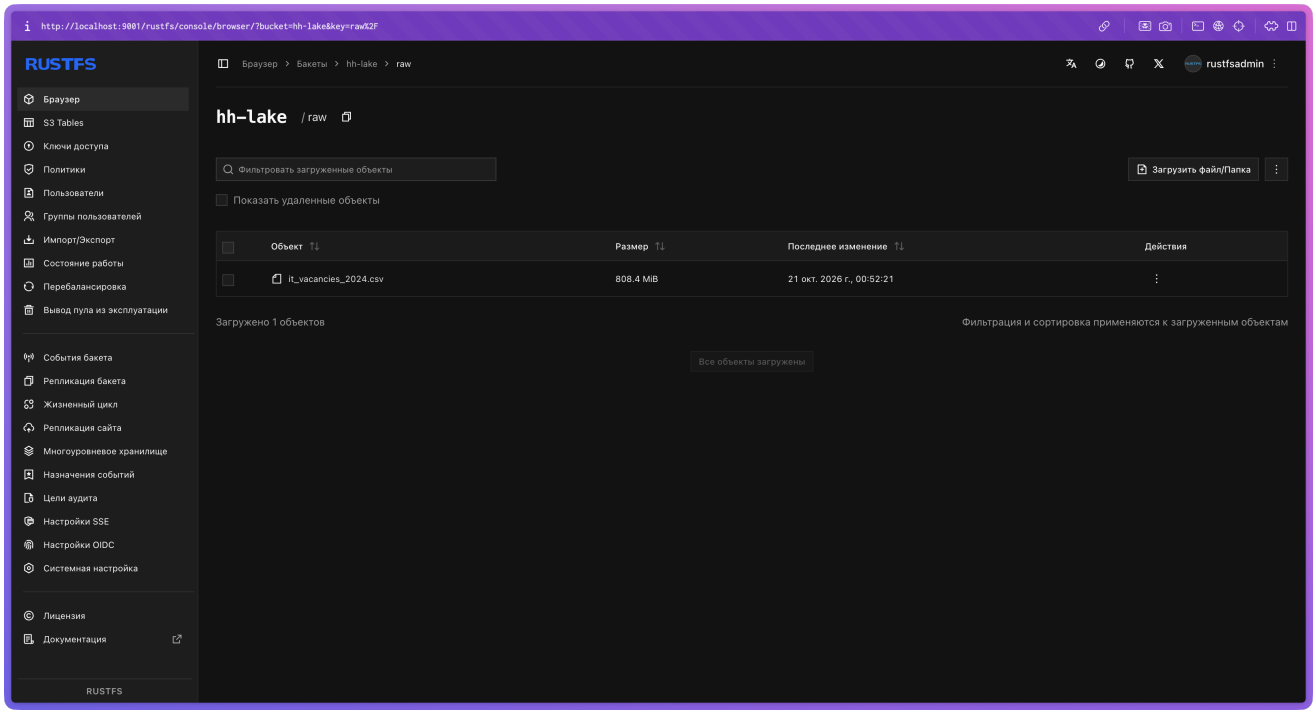



**Рис. 2:** Бакет hh-lake в веб-консоли RustFS: объект raw/it_vacancies_2024.csv

## 3.1 Правила обработки, общие для всех способов

Чтобы два способа можно было честно сравнить, правила зафиксированы заранее. В Python и в SQL они написаны независимо друг от друга.

- **Навыки.** Поле `key_skills` делится по «;». Каждый навык обрезается от пробелов, табуляций и переводов строк и приводится к нижнему регистру, пустые отбрасываются, повторы внутри вакансии убираются. Синонимы (например, «c#» и «c sharp») я не склеиваю: это уже интерпретация, а не подсчёт.
- **Сочетания.** Это упорядоченные по алфавиту кортежи из $k$ разных навыков одной вакансии. Вакансия с $n$ навыками даёт ровно $\binom{n}{k}$ сочетаний размера $k$.

- **Зарплата.** Берётся только валюта RUR. Если указаны обе границы вилки, беру середину, если одна — её. Если в вакансии указано «до вычета налогов» (`gross = true`), умножаю на 0,87, чтобы всё было «на руки». Значения вне диапазона 10 000–1 000 000 руб. считаю мусором.

```
def is_variant(row):
    if (row.get("role") or "") != VARIANT_ROLE:
        return False
    name = (row.get("name") or "").lower()
    return any(token in name for token in VARIANT_TOKENS)
```

```
def parse_skills(text):
    found = set()
    for part in (text or "").split(";"):
        skill = part.strip(SKILL_STRIP).lower()
        if skill:
            found.add(skill)
    return sorted(found)
```

# 4 Этап 3а. Свой MapReduce из S3

## 4.1 Как устроен

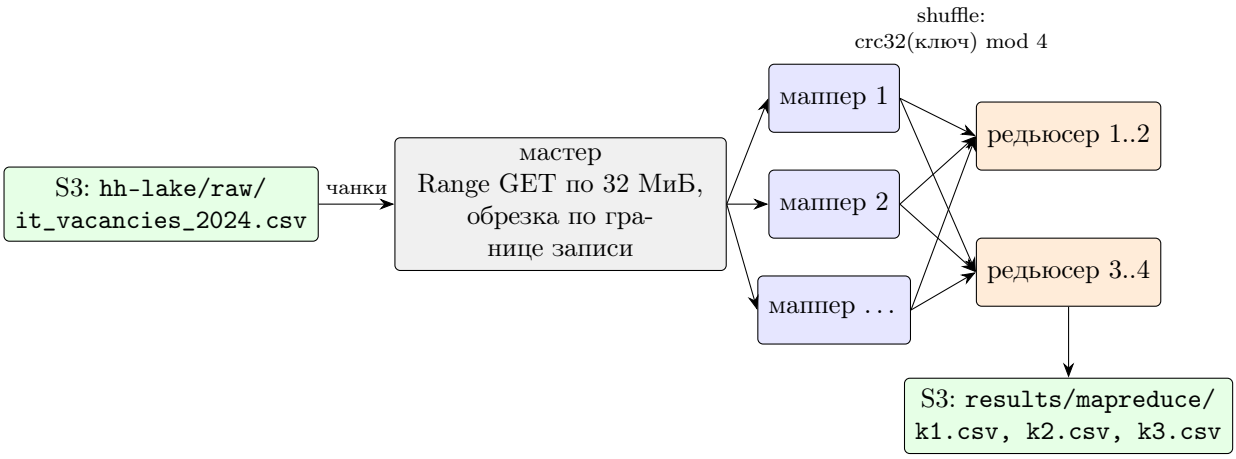



**Рис. 3:** Схема моего MapReduce

1. **Чтение чанками.** Мастер узнаёт размер объекта запросом HEAD и читает его кусками по 32 МиБ запросами `GET` с заголовком `Range: bytes=a-b`. Объект целиком никогда не лежит в памяти. У меня вышло 26 запросов и 26 map-задач.
2. **Граница записи.** Обычный разрез по байтам режет строку CSV пополам. Хуже того, в поле описания вакансии могут быть переводы строк внутри кавычек. Поэтому конец последней целой записи в чанке ищется так: идём от конца по символам «\n» и берём первый, перед которым стоит *чётное* число кавычек. Нечётное число значит, что мы внутри кавычек. Хвост после этой границы переносится в начало следующего чанка. Для экранированной кавычки  это тоже работает: она добавляет две кавычки и чётность не меняет.
3. **Map.** Каждая задача уходит в отдельный процесс. Маппер разбирает CSV, отбирает вакансии варианта, нормализует навыки и порождает все сочетания $k = 1, 2, 3$.
4. **Combiner.** Внутри маппера одинаковые ключи сразу складываются в счётчик, и в shuffle уходит пара (ключ, частота в чанке). Без этого в shuffle ушли бы 426 634 записей вида (ключ, 1), а с ним уходит 287 150, то есть на 32,7 % меньше.
5. **Shuffle и Reduce.** Ключ отправляется редьюсеру с номером crc32(ключ) mod 4, поэтому одинаковые ключи из всех чанков попадают к одному редьюсеру. Редьюсеры работают тоже в отдельных процессах и складывают частоты. В наивной версии редьюсер честно сортирует пары (ключ, 1) и суммирует группы, как в классическом MapReduce.

6. **Контроль.** В конце проверяется $\sum \text{частот}_k = \sum_i \binom{n_i}{k}$ по всем вакансиям. Если равенство не выполняется, программа падает. Результаты k1–k3 записываются в бакет и для проверки сразу читаются обратно.

```
def record_boundary(buf):
    total = buf.count(b'"')
    pos = len(buf)
    while True:
        pos = buf.rfind(b"\n", 0, pos)
        if pos < 0:
            return -1
        if (total - buf.count(b'"', pos)) % 2 == 0:
            return pos + 1
```

```
def map_task(task_id, attempt, data, fields, mode, fail_task, reducers):
    if task_id == fail_task and attempt == 1:
        print(f"  [маппер задачи {task_id}] имитирую падение процесса: os._exit(137)\n", end="",
        ↪  flush=True)
        os._exit(137)
    t0 = time.perf_counter()
    parser = "csv"
    if mode == "opt":
        try:
            rows = list(rows_arrow(data, fields))
            parser = "arrow"
        except Exception:
            rows = list(rows_csv(data, fields))
            parser = "csv (запасной)"
    else:
        rows = rows_csv(data, fields)
    if mode == "naive":
        parts = [[[] for _ in c.KS] for _ in range(reducers)]
    else:
        parts = [[Counter() for _ in c.KS] for _ in range(reducers)]
    scanned = 0
    vacancies = []
    emitted = [0, 0, 0]
    for row in rows:
        scanned += 1
        if row is None or not c.is_variant(row):
            continue
        skills = c.parse_skills(row.get("key_skills"))
        vacancies.append(c.vacancy_record(row, skills))
        for k in c.KS:
            for combo in combinations(skills, k):
                emitted[k - 1] += 1
                target = parts[partition_of(combo, reducers)][k - 1]
                if mode == "naive":
                    target.append((combo, 1))
                else:
                    target[combo] += 1
    shuffled = sum(len(parts[r][k - 1]) for r in range(reducers) for k in c.KS)
    return {"task": task_id, "attempt": attempt, "pid": os.getpid(), "parser": parser, "scanned":
    ↪  scanned,
            "vacancies": vacancies, "emitted": emitted, "shuffled": shuffled, "parts": parts,
            "seconds": time.perf_counter() - t0, "bytes": len(data)}
```

```
def reduce_task(r, k, pieces, mode):
    if mode == "naive":
        pairs = [p for piece in pieces for p in piece]
        pairs.sort()
        total = {}
        prev = None
        acc = 0
        for combo, one in pairs:
            if combo != prev:
```

```
                if prev is not None:
                    total[prev] = acc
                prev, acc = combo, 0
            acc += one
        if prev is not None:
            total[prev] = acc
        return r, k, total
    total = Counter()
    for piece in pieces:
        total.update(piece)
    return r, k, dict(total)
```

## 4.2 Запуск

Оптимизированная версия: 4 маппера, 4 редьюсера, разбор CSV через pyarrow. Время — 5,80 с, из них 3,99 с ушло на чтение из S3. Тут же запускается наивная версия. Её результаты побайтно совпали с оптимизированной, сверка идёт по SHA-256 файлов k1–k3.

```
==== этап 3a. свой mapreduce: читает объект из s3 чанками (range get) и считает навыки, пары и тройки ====
вариант: программист, разработчик на c# и .net
условие отбора: role = 'Программист, разработчик' и в названии вакансии есть c#, c#, .net, dotnet, c sharp,
↪  csharp или си шарп

доказательство, что вычисления не видят локальный файл: контейнер с python получает только код и папку для
↪  результатов

$ (внутри контейнера) ls /app; ls /data
lab
report
results
  ls: cannot access '/data': No such file or directory
  -> папки /data с датасетом в контейнере нет, данные можно взять только из s3

$ python -m lab.mapreduce --mode opt --verbose
объект s3://hh-lake/raw/it_vacancies_2024.csv: 847619384 байт (808.35 миб)
режим opt: чанк 32 миб (около 26 range get), мапперов 4, редьюсеров 4
  map   0: попытка 1, процесс 8, 32.00 миб, строк 8968, вакансий варианта 149, парсер arrow, 0.27 с
  map   2: попытка 1, процесс 8, 32.00 миб, строк 9059, вакансий варианта 173, парсер arrow, 0.11 с
  map   5: попытка 1, процесс 8, 32.00 миб, строк 9288, вакансий варианта 189, парсер arrow, 0.13 с
  map   1: попытка 1, процесс 11, 32.00 миб, строк 9119, вакансий варианта 138, парсер arrow, 0.27 с
  map   4: попытка 1, процесс 11, 32.00 миб, строк 9305, вакансий варианта 174, парсер arrow, 0.14 с
  map   3: попытка 1, процесс 12, 32.00 миб, строк 9114, вакансий варианта 163, парсер arrow, 0.25 с
  map   9: попытка 1, процесс 8, 32.00 миб, строк 9310, вакансий варианта 137, парсер arrow, 0.09 с
  map   6: попытка 1, процесс 13, 32.00 миб, строк 9178, вакансий варианта 205, парсер arrow, 0.27 с
  map  12: попытка 1, процесс 11, 32.00 миб, строк 9033, вакансий варианта 170, парсер arrow, 0.10 с
  map   7: попытка 1, процесс 12, 32.00 миб, строк 9063, вакансий варианта 154, парсер arrow, 0.10 с
  map  11: попытка 1, процесс 12, 32.00 миб, строк 9123, вакансий варианта 194, парсер arrow, 0.11 с
  map  10: попытка 1, процесс 13, 32.00 миб, строк 9220, вакансий варианта 168, парсер arrow, 0.10 с
  map   8: попытка 1, процесс 11, 32.00 миб, строк 9109, вакансий варианта 183, парсер arrow, 0.10 с
  map  16: попытка 1, процесс 11, 32.00 миб, строк 8885, вакансий варианта 172, парсер arrow, 0.11 с
  map  19: попытка 1, процесс 12, 32.00 миб, строк 9202, вакансий варианта 197, парсер arrow, 0.11 с
  map  17: попытка 1, процесс 8, 32.00 миб, строк 8895, вакансий варианта 199, парсер arrow, 0.12 с
  map  14: попытка 1, процесс 13, 32.00 миб, строк 9071, вакансий варианта 136, парсер arrow, 0.11 с
  map  15: попытка 1, процесс 12, 32.00 миб, строк 9474, вакансий варианта 201, парсер arrow, 0.11 с
  map  18: попытка 1, процесс 13, 32.00 миб, строк 9135, вакансий варианта 175, парсер arrow, 0.10 с
  map  13: попытка 1, процесс 8, 32.00 миб, строк 9107, вакансий варианта 166, парсер arrow, 0.09 с
  map  22: попытка 1, процесс 13, 32.00 миб, строк 8817, вакансий варианта 199, парсер arrow, 0.11 с
  map  20: попытка 1, процесс 11, 32.00 миб, строк 8819, вакансий варианта 160, парсер arrow, 0.09 с
  map  21: попытка 1, процесс 8, 32.00 миб, строк 8973, вакансий варианта 163, парсер arrow, 0.10 с
  map  23: попытка 1, процесс 12, 32.00 миб, строк 8876, вакансий варианта 183, парсер arrow, 0.11 с
  map  25: попытка 1, процесс 8, 8.35 миб, строк 2305, вакансий варианта 32, парсер arrow, 0.03 с
  map  24: попытка 1, процесс 11, 32.00 миб, строк 8940, вакансий варианта 176, парсер arrow, 0.09 с
прочитано из s3: 847619384 байт за 26 range get, это весь объект: да
map-задач: 26, просмотрено записей: 229388, вакансий варианта: 4356 (с навыками 3345, с зарплатой 1442)
записей ушло в shuffle: 287150 (до combiner было бы 426634)
k=1: уникальных сочетаний 1164, сумма частот 22866, сумма C(n,1) по вакансиям 22866, совпадает: да; топ: c# =
↪  2237
k=2: уникальных сочетаний 17198, сумма частот 92865, сумма C(n,2) по вакансиям 92865, совпадает: да; топ: .net
↪  framework + c# = 793
k=3: уникальных сочетаний 114332, сумма частот 310903, сумма C(n,3) по вакансиям 310903, совпадает: да; топ:
↪  asp.net + c# + postgresql = 266
```

```
время: всего 5.80 с, map 5.56 с (из них чтение s3 3.99 с), reduce 0.25 с
  сохранено s3://hh-lake/results/mapreduce/k1.csv (23779 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/mapreduce/k2.csv (482273 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/mapreduce/k3.csv (4142254 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/mapreduce/vacancies.jsonl (1264674 байт), sha256 после чтения обратно совпал:
  ↪  да
  сохранено s3://hh-lake/results/mapreduce/summary.json (652 байт), sha256 после чтения обратно совпал: да

==== та же задача наивной версией (без combiner, стандартный csv, один маппер) - для сверки и бонуса ====

$ python -m lab.mapreduce --mode naive --workers 1 --tag mapreduce_naive --compare-with mapreduce
объект s3://hh-lake/raw/it_vacancies_2024.csv: 847619384 байт (808.35 миб)
режим naive: чанк 32 миб (около 26 range get), мапперов 1, редьюсеров 4
прочитано из s3: 847619384 байт за 26 range get, это весь объект: да
map-задач: 26, просмотрено записей: 229388, вакансий варианта: 4356 (с навыками 3345, с зарплатой 1442)
записей ушло в shuffle: 426634 (до combiner было бы 426634)
k=1: уникальных сочетаний 1164, сумма частот 22866, сумма C(n,1) по вакансиям 22866, совпадает: да; топ: c# =
↪  2237
k=2: уникальных сочетаний 17198, сумма частот 92865, сумма C(n,2) по вакансиям 92865, совпадает: да; топ: .net
↪  framework + c# = 793
k=3: уникальных сочетаний 114332, сумма частот 310903, сумма C(n,3) по вакансиям 310903, совпадает: да; топ:
↪  asp.net + c# + postgresql = 266
время: всего 13.84 с, map 12.78 с (из них чтение s3 2.63 с), reduce 1.06 с
  сохранено s3://hh-lake/results/mapreduce_naive/k1.csv (23779 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/mapreduce_naive/k2.csv (482273 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/mapreduce_naive/k3.csv (4142254 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/mapreduce_naive/vacancies.jsonl (1264674 байт), sha256 после чтения обратно
  ↪  совпал: да
  сохранено s3://hh-lake/results/mapreduce_naive/summary.json (656 байт), sha256 после чтения обратно совпал: да
k1.csv: sha256 cd0880521cf20e6b..., эталон results/mapreduce: cd0880521cf20e6b..., совпадает: да
k2.csv: sha256 ce075fd4e1d7d5a5..., эталон results/mapreduce: ce075fd4e1d7d5a5..., совпадает: да
k3.csv: sha256 a0fd213179d8fcec..., эталон results/mapreduce: a0fd213179d8fcec..., совпадает: да
результат совпал с эталонным прогоном: да
```

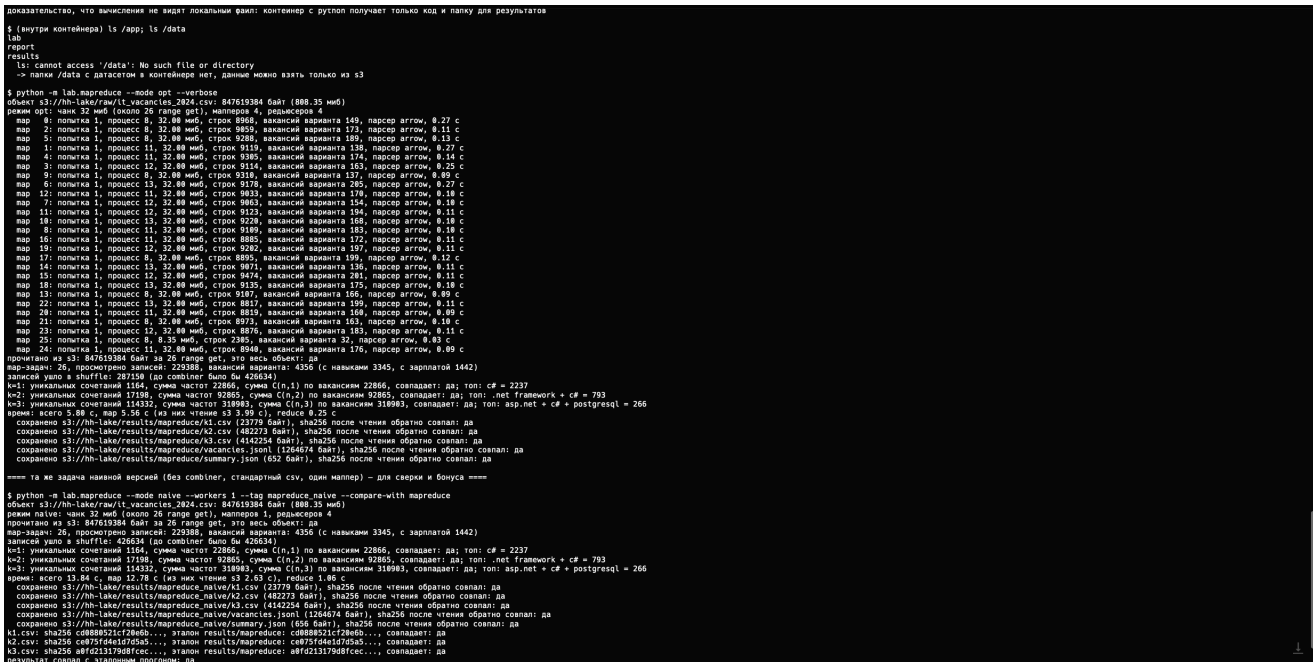



**Рис. 4:** Работа MapReduce: map-задачи, контроль сумм, запись результатов в S3

## 4.3 Отказ маппера

Падение процесса я имитирую по-настоящему. На первой попытке задачи 5 маппер вызывает `os._exit(137)`, то есть процесс умирает так же, как от нехватки памяти (сигнал 9 даёт тот же код 137). Пул процессов после этого становится сломанным: все незавершённые задачи получают `BrokenProcessPool`. Мастер ловит это исключение, пересоздаёт пул и заново отправляет все задачи, результат которых ещё не получен. Данные этих задач он держит у себя до подтверждения.

В итог попадает только результат успешной попытки, поэтому двойного счёта нет. Пересозданий пула: 1, повторно запущены задачи: 3 (попытка 2), 4 (попытка 2), 5 (попытка 2), 6 (попытка 2), 7 (попытка 2), 8 (попытка 2), 9 (попытка 2). Результаты с отказом и без отказа совпали.

```
    def recover(self, reason, extra=()):
        lost = sorted(set(self.futures.values()) | set(extra))
        self.pool.shutdown(wait=False, cancel_futures=True)
        self.futures.clear()
        self.restarts += 1
        print(f"  мастер: {reason}. пул процессов пересоздан, задачи перезапускаются: {lost}",
        ↪  flush=True)
        self.pool = ProcessPoolExecutor(max_workers=self.workers, mp_context=self.ctx)
        for tid in lost:
            self.tasks[tid][1] += 1
            if self.tasks[tid][1] > MAX_ATTEMPTS:
                raise RuntimeError(f"задача {tid} упала {MAX_ATTEMPTS} раза подряд")
            self.retried.append((tid, self.tasks[tid][1]))
            self.submit(tid)
```

```
==== проверка отказоустойчивости: процесс-маппер задачи 5 умирает (os._exit(137)) на первой попытке ====
мастер должен заметить падение, пересоздать пул процессов, перезапустить потерянные задачи
и получить ровно тот же результат, что и без сбоя (сравниваем sha256 файлов k1..k3)

$ python -m lab.mapreduce --mode opt --fail-task 5 --verbose --tag mapreduce_failtest --compare-with mapreduce
объект s3://hh-lake/raw/it_vacancies_2024.csv: 847619384 байт (808.35 миб)
режим opt: чанк 32 миб (около 26 range get), мапперов 4, редьюсеров 4, сбой маппера задачи 5
  map   1: попытка 1, процесс 11, 32.00 миб, строк 9119, вакансий варианта 138, парсер arrow, 0.30 с
  map   0: попытка 1, процесс 8, 32.00 миб, строк 8968, вакансий варианта 149, парсер arrow, 0.31 с
  map   2: попытка 1, процесс 8, 32.00 миб, строк 9059, вакансий варианта 173, парсер arrow, 0.14 с
  [маппер задачи 5] имитирую падение процесса: os._exit(137)
  мастер: пул процессов сломан, один из мапперов аварийно завершился. пул процессов пересоздан, задачи
  ↪  перезапускаются: [3, 4, 5, 6, 7, 8, 9]
  map   3: попытка 2, процесс 61, 32.00 миб, строк 9114, вакансий варианта 163, парсер arrow, 0.24 с
  map   4: попытка 2, процесс 62, 32.00 миб, строк 9305, вакансий варианта 174, парсер arrow, 0.33 с
  map   7: попытка 2, процесс 61, 32.00 миб, строк 9063, вакансий варианта 154, парсер arrow, 0.09 с
  map   5: попытка 2, процесс 63, 32.00 миб, строк 9288, вакансий варианта 189, парсер arrow, 0.32 с
  map   6: попытка 2, процесс 58, 32.00 миб, строк 9178, вакансий варианта 205, парсер arrow, 0.26 с
  map  11: попытка 1, процесс 58, 32.00 миб, строк 9123, вакансий варианта 194, парсер arrow, 0.10 с
  map   8: попытка 2, процесс 62, 32.00 миб, строк 9109, вакансий варианта 183, парсер arrow, 0.13 с
  map   9: попытка 2, процесс 63, 32.00 миб, строк 9310, вакансий варианта 137, парсер arrow, 0.11 с
  map  12: попытка 1, процесс 62, 32.00 миб, строк 9033, вакансий варианта 170, парсер arrow, 0.11 с
  map  10: попытка 1, процесс 61, 32.00 миб, строк 9220, вакансий варианта 168, парсер arrow, 0.12 с
  map  13: попытка 1, процесс 63, 32.00 миб, строк 9107, вакансий варианта 166, парсер arrow, 0.13 с
  map  18: попытка 1, процесс 61, 32.00 миб, строк 9135, вакансий варианта 175, парсер arrow, 0.13 с
  map  16: попытка 1, процесс 62, 32.00 миб, строк 8885, вакансий варианта 172, парсер arrow, 0.13 с
  map  15: попытка 1, процесс 58, 32.00 миб, строк 9474, вакансий варианта 201, парсер arrow, 0.17 с
  map  14: попытка 1, процесс 61, 32.00 миб, строк 9071, вакансий варианта 136, парсер arrow, 0.11 с
  map  17: попытка 1, процесс 63, 32.00 миб, строк 8895, вакансий варианта 199, парсер arrow, 0.13 с
  map  19: попытка 1, процесс 58, 32.00 миб, строк 9202, вакансий варианта 197, парсер arrow, 0.12 с
  map  21: попытка 1, процесс 63, 32.00 миб, строк 8973, вакансий варианта 163, парсер arrow, 0.10 с
  map  22: попытка 1, процесс 61, 32.00 миб, строк 8817, вакансий варианта 199, парсер arrow, 0.12 с
  map  20: попытка 1, процесс 62, 32.00 миб, строк 8819, вакансий варианта 160, парсер arrow, 0.11 с
  map  23: попытка 1, процесс 58, 32.00 миб, строк 8876, вакансий варианта 183, парсер arrow, 0.11 с
  map  25: попытка 1, процесс 63, 8.35 миб, строк 2305, вакансий варианта 32, парсер arrow, 0.03 с
  map  24: попытка 1, процесс 62, 32.00 миб, строк 8940, вакансий варианта 176, парсер arrow, 0.10 с
прочитано из s3: 847619384 байт за 26 range get, это весь объект: да
map-задач: 26, просмотрено записей: 229388, вакансий варианта: 4356 (с навыками 3345, с зарплатой 1442)
записей ушло в shuffle: 287150 (до combiner было бы 426634)
k=1: уникальных сочетаний 1164, сумма частот 22866, сумма C(n,1) по вакансиям 22866, совпадает: да; топ: c# =
↪  2237
k=2: уникальных сочетаний 17198, сумма частот 92865, сумма C(n,2) по вакансиям 92865, совпадает: да; топ: .net
↪  framework + c# = 793
k=3: уникальных сочетаний 114332, сумма частот 310903, сумма C(n,3) по вакансиям 310903, совпадает: да; топ:
↪  asp.net + c# + postgresql = 266
перезапусков пула: 1, повторных попыток задач: 7 [(3, 2), (4, 2), (5, 2), (6, 2), (7, 2), (8, 2), (9, 2)]
время: всего 6.34 с, map 6.10 с (из них чтение s3 3.50 с), reduce 0.24 с
  сохранено s3://hh-lake/results/mapreduce_failtest/k1.csv (23779 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/mapreduce_failtest/k2.csv (482273 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/mapreduce_failtest/k3.csv (4142254 байт), sha256 после чтения обратно совпал:
  ↪  да
  сохранено s3://hh-lake/results/mapreduce_failtest/vacancies.jsonl (1264674 байт), sha256 после чтения обратно
  ↪  совпал: да
```

```
  сохранено s3://hh-lake/results/mapreduce_failtest/summary.json (863 байт), sha256 после чтения обратно совпал:
  ↪  да
k1.csv: sha256 cd0880521cf20e6b..., эталон results/mapreduce: cd0880521cf20e6b..., совпадает: да
k2.csv: sha256 ce075fd4e1d7d5a5..., эталон results/mapreduce: ce075fd4e1d7d5a5..., совпадает: да
k3.csv: sha256 a0fd213179d8fcec..., эталон results/mapreduce: a0fd213179d8fcec..., совпадает: да
результат совпал с эталонным прогоном: да
```

```
==== проверка отказоустойчивости: процесс-маппер задачи 5 умирает (os._exit(137)) на первой попытке ====
мастер должен заметить падение, пересоздать пул процессов, перезапустить потерянные задачи
и получить ровно тот же результат, что и без сбоя (сравниваем sha256 файлов k1..k3)

$ python -m lab.mapreduce --mode opt --fail-task 5 --verbose --tag mapreduce_failtest --compare-with mapreduce
объект s3://hh-lake/raw/it_vacancies_2024.csv: 847619384 байт (808.35 миб)
режим opt: чанк 32 миб (около 26 range get), мапперов 4, редьюсеров 4, сбой маппера задачи 5
  map   1: попытка 1, процесс 11, 32.00 миб, строк 9119, вакансий варианта 138, парсер arrow, 0.30 с
  map   0: попытка 1, процесс 8, 32.00 миб, строк 8968, вакансий варианта 149, парсер arrow, 0.31 с
  map   2: попытка 1, процесс 8, 32.00 миб, строк 9059, вакансий варианта 173, парсер arrow, 0.14 с
  [маппер задачи 5] имитирую падение процесса: os._exit(137)
  мастер: пул процессов сломан, один из мапперов аварийно завершился. пул процессов пересоздан, задачи перезапускаются: [3,
4, 5, 6, 7, 8, 9]
  map   3: попытка 2, процесс 61, 32.00 миб, строк 9114, вакансий варианта 163, парсер arrow, 0.24 с
  map   4: попытка 2, процесс 62, 32.00 миб, строк 9305, вакансий варианта 174, парсер arrow, 0.33 с
  map   7: попытка 2, процесс 61, 32.00 миб, строк 9063, вакансий варианта 154, парсер arrow, 0.09 с
  map   5: попытка 2, процесс 63, 32.00 миб, строк 9288, вакансий варианта 189, парсер arrow, 0.32 с
  map   6: попытка 2, процесс 58, 32.00 миб, строк 9178, вакансий варианта 205, парсер arrow, 0.26 с
  map  11: попытка 1, процесс 58, 32.00 миб, строк 9123, вакансий варианта 194, парсер arrow, 0.10 с
  map   8: попытка 2, процесс 62, 32.00 миб, строк 9109, вакансий варианта 183, парсер arrow, 0.13 с
  map   9: попытка 2, процесс 63, 32.00 миб, строк 9310, вакансий варианта 137, парсер arrow, 0.11 с
  map  12: попытка 1, процесс 62, 32.00 миб, строк 9033, вакансий варианта 170, парсер arrow, 0.11 с
  map  10: попытка 1, процесс 61, 32.00 миб, строк 9220, вакансий варианта 168, парсер arrow, 0.12 с
  map  13: попытка 1, процесс 63, 32.00 миб, строк 9107, вакансий варианта 166, парсер arrow, 0.13 с
  map  18: попытка 1, процесс 61, 32.00 миб, строк 9135, вакансий варианта 175, парсер arrow, 0.13 с
  map  16: попытка 1, процесс 62, 32.00 миб, строк 8885, вакансий варианта 172, парсер arrow, 0.13 с
  map  15: попытка 1, процесс 58, 32.00 миб, строк 9474, вакансий варианта 201, парсер arrow, 0.17 с
  map  14: попытка 1, процесс 61, 32.00 миб, строк 9071, вакансий варианта 136, парсер arrow, 0.11 с
  map  17: попытка 1, процесс 63, 32.00 миб, строк 8895, вакансий варианта 199, парсер arrow, 0.13 с
  map  19: попытка 1, процесс 58, 32.00 миб, строк 9202, вакансий варианта 197, парсер arrow, 0.12 с
  map  21: попытка 1, процесс 63, 32.00 миб, строк 8973, вакансий варианта 163, парсер arrow, 0.10 с
  map  22: попытка 1, процесс 61, 32.00 миб, строк 8817, вакансий варианта 199, парсер arrow, 0.12 с
  map  20: попытка 1, процесс 62, 32.00 миб, строк 8819, вакансий варианта 160, парсер arrow, 0.11 с
  map  23: попытка 1, процесс 58, 32.00 миб, строк 8876, вакансий варианта 183, парсер arrow, 0.11 с
  map  25: попытка 1, процесс 63, 8.35 миб, строк 2305, вакансий варианта 32, парсер arrow, 0.03 с
  map  24: попытка 1, процесс 62, 32.00 миб, строк 8940, вакансий варианта 176, парсер arrow, 0.10 с
прочитано из s3: 847619384 байт за 26 range get, это весь объект: да
map-задач: 26, просмотрено записей: 229388, вакансий варианта: 4356 (с навыками 3345, с зарплатой 1442)
записей ушло в shuffle: 287150 (до combiner было бы 426634)
k=1: уникальных сочетаний 1164, сумма частот 22866, сумма C(n,1) по вакансиям 22866, совпадает: да; топ: c# = 2237
k=2: уникальных сочетаний 17198, сумма частот 92865, сумма C(n,2) по вакансиям 92865, совпадает: да; топ: .net framework + c
# = 793
k=3: уникальных сочетаний 114332, сумма частот 310903, сумма C(n,3) по вакансиям 310903, совпадает: да; топ: asp.net + c# +
postgresql = 266
перезапусков пула: 1, повторных попыток задач: 7 [(3, 2), (4, 2), (5, 2), (6, 2), (7, 2), (8, 2), (9, 2)]
время: всего 6.34 с, map 6.10 с (из них чтение s3 3.50 с), reduce 0.24 с
  сохранено s3://hh-lake/results/mapreduce_failtest/k1.csv (23779 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/mapreduce_failtest/k2.csv (482273 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/mapreduce_failtest/k3.csv (4142254 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/mapreduce_failtest/vacancies.jsonl (1264674 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/mapreduce_failtest/summary.json (863 байт), sha256 после чтения обратно совпал: да
k1.csv: sha256 cd0880521cf20e6b..., эталон results/mapreduce: cd0880521cf20e6b..., совпадает: да
k2.csv: sha256 ce075fd4e1d7d5a5..., эталон results/mapreduce: ce075fd4e1d7d5a5..., совпадает: да
k3.csv: sha256 a0fd213179d8fcec..., эталон results/mapreduce: a0fd213179d8fcec..., совпадает: да
результат совпал с эталонным прогоном: да
```

**Рис. 5:** Отказ маппера: падение процесса, перезапуск задач, совпадение результата

# 5 Этап 3б. Тот же подсчёт в DuckDB

DuckDB читает тот же объект прямо из S3 через расширение `httpfs`. Для этого создаётся секрет типа s3 с адресом `rustfs:9000` и адресацией по пути (`URL_STYLE path`). Ничего заранее не скачивается. Условие варианта я написал в SQL заново, а не скопировал из Python: в SQL это `regexp_matches` по регулярному выражению, в Python — проверка подстрок. Пары и тройки получаются самосоединением таблицы «вакансия–навык» с условием $a < b < c$. Это то же самое, что упорядоченные сочетания в Python.

```
VARIANT_SQL = r"""
    role = 'Программист, разработчик'
    AND regexp_matches(lower(name), 'c#|c#|\.net|dotnet|c sharp|csharp|си шарп')
"""
```

```
SKILLS_SQL = r"""
    list_sort(list_distinct(list_filter(
        list_transform(string_split(coalesce(key_skills, ''), ';'),
                       s -> lower(trim(s, ' ' || chr(9) || chr(13) || chr(10)))),
        s -> s <> '')))
"""
```

```
COUNT_SQL = {
    1: """SELECT skill AS skill_1, count(*) AS cnt FROM s GROUP BY ALL""",
```

```
    2: """SELECT a.skill AS skill_1, b.skill AS skill_2, count(*) AS cnt
          FROM s a JOIN s b ON a.rid = b.rid AND a.skill < b.skill GROUP BY ALL""",
    3: """SELECT a.skill AS skill_1, b.skill AS skill_2, c.skill AS skill_3, count(*) AS cnt
          FROM s a JOIN s b ON a.rid = b.rid AND a.skill < b.skill
                   JOIN s c ON a.rid = c.rid AND b.skill < c.skill GROUP BY ALL""",
}
```

```
==== этап 3б. тот же подсчёт sql-запросом duckdb прямо по объекту в s3 ====
условие варианта в sql написано отдельно от python:
  where role = 'Программист, разработчик'
    and regexp_matches(lower(name), 'c#|c#|\.net|dotnet|c sharp|csharp|си шарп')

$ python -m lab.duck --source csv
доступ к s3 из duckdb: httpfs, источник: csv, потоков 4, лимит памяти 1500MB
вакансий варианта: 4356 (с навыками 3345)
k=1: уникальных сочетаний 1164, сумма частот 22866; топ: c# = 2237
k=2: уникальных сочетаний 17198, сумма частот 92865; топ: .net framework + c# = 793
k=3: уникальных сочетаний 114332, сумма частот 310903; топ: asp.net + c# + postgresql = 266
время: всего 4.40 с, из них чтение и фильтр 4.13 с
  сохранено s3://hh-lake/results/duckdb/k1.csv (23779 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/duckdb/k2.csv (482273 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/duckdb/k3.csv (4142254 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/duckdb/vacancy_ids.txt (41014 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/duckdb/summary.json (342 байт), sha256 после чтения обратно совпал: да
```

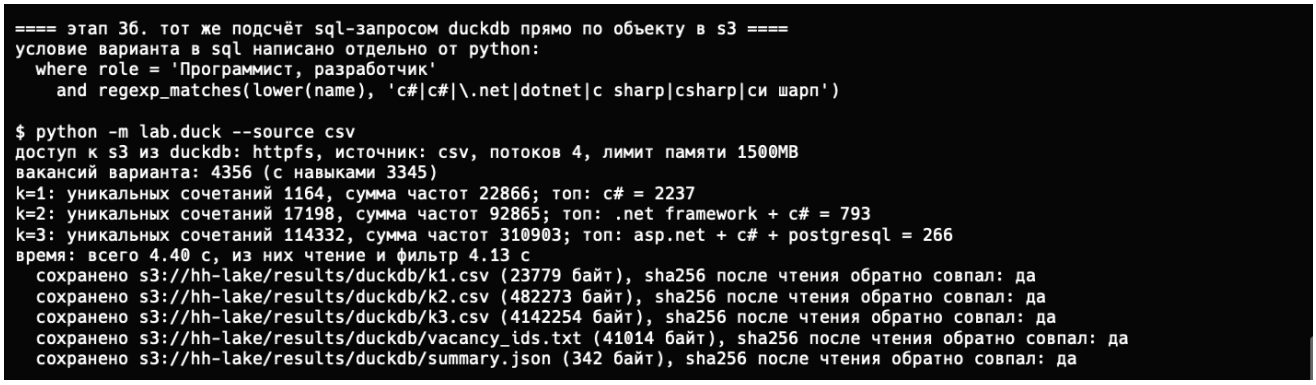



**Рис. 6:** Подсчёт навыков, пар и троек в DuckDB

# 6 Этап 4. Сверка и рост числа сочетаний

## 6.1 Совпадение результатов

Скрипт `verify.py` читает результаты всех способов из S3. Сравниваются не только итоговые суммы, но и каждый ключ: какие сочетания есть только у одного способа и у каких одинаковых ключей разные частоты. Отдельно сравнивается мультимножество id отобранных вакансий. Если фильтр в SQL и Python разошёлся хотя бы на одну вакансию, это всплывёт сразу.

Итог: результаты всех 5 способов **полностью совпали**. Всего в датасете 229 388 записей, в вариант попало 4 356 вакансий, из них 3 345 с навыками и 1 442 с зарплатой в рублях.

| $k$ | Способ | Ключей | $\sum$ частот | Самое частое сочетание | Частота | Сверка |
|---|---|---|---|---|---|---|
| k=1 | mapreduce (основной) | 1 164 | 22 866 | c# | 2 237 | эталон |
| k=1 | duckdb, csv | 1 164 | 22 866 | c# | 2 237 | совпало |
| k=1 | mapreduce, наивный | 1 164 | 22 866 | c# | 2 237 | совпало |
| k=1 | mapreduce с отказом маппера | 1 164 | 22 866 | c# | 2 237 | совпало |
| k=1 | duckdb, parquet | 1 164 | 22 866 | c# | 2 237 | совпало |
| k=2 | mapreduce (основной) | 17 198 | 92 865 | .net framework + c# | 793 | эталон |
| k=2 | duckdb, csv | 17 198 | 92 865 | .net framework + c# | 793 | совпало |
| k=2 | mapreduce, наивный | 17 198 | 92 865 | .net framework + c# | 793 | совпало |
| k=2 | mapreduce с отказом маппера | 17 198 | 92 865 | .net framework + c# | 793 | совпало |
| k=2 | duckdb, parquet | 17 198 | 92 865 | .net framework + c# | 793 | совпало |
| k=3 | mapreduce (основной) | 114 332 | 310 903 | asp.net + c# + postgresql | 266 | эталон |
| k=3 | duckdb, csv | 114 332 | 310 903 | asp.net + c# + postgresql | 266 | совпало |
| k=3 | mapreduce, наивный | 114 332 | 310 903 | asp.net + c# + postgresql | 266 | совпало |
| k=3 | mapreduce с отказом маппера | 114 332 | 310 903 | asp.net + c# + postgresql | 266 | совпало |
| k=3 | duckdb, parquet | 114 332 | 310 903 | asp.net + c# + postgresql | 266 | совпало |

**Таблица 2:** Контрольные числа всех способов

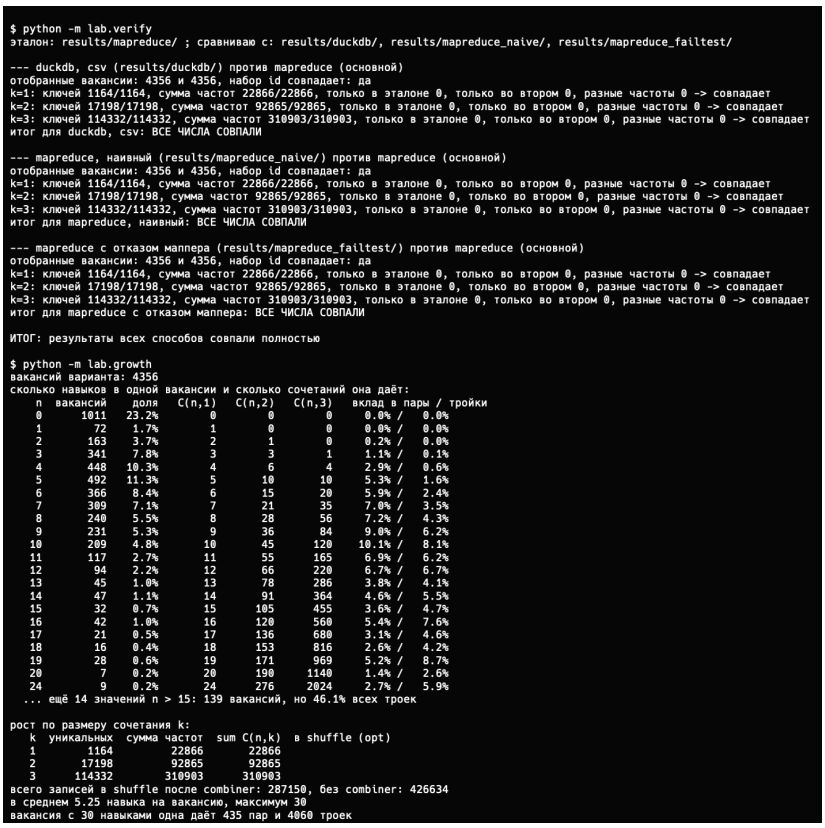
```
$ python -m lab.verify
эталон: results/mapreduce/ ; сравниваю с: results/duckdb/, results/mapreduce_naive/, results/mapreduce_failtest/

--- duckdb, csv (results/duckdb/) против mapreduce (основной)
отобранные вакансии: 4356 и 4356, набор id совпадает: да
k=1: ключей 1164/1164, сумма частот 22866/22866, только в эталоне 0, только во втором 0, разные частоты 0 ->
↪  совпадает
k=2: ключей 17198/17198, сумма частот 92865/92865, только в эталоне 0, только во втором 0, разные частоты 0 ->
↪  совпадает
k=3: ключей 114332/114332, сумма частот 310903/310903, только в эталоне 0, только во втором 0, разные частоты 0
↪  -> совпадает
итог для duckdb, csv: ВСЕ ЧИСЛА СОВПАЛИ

--- mapreduce, наивный (results/mapreduce_naive/) против mapreduce (основной)
отобранные вакансии: 4356 и 4356, набор id совпадает: да
k=1: ключей 1164/1164, сумма частот 22866/22866, только в эталоне 0, только во втором 0, разные частоты 0 ->
↪  совпадает
k=2: ключей 17198/17198, сумма частот 92865/92865, только в эталоне 0, только во втором 0, разные частоты 0 ->
↪  совпадает
k=3: ключей 114332/114332, сумма частот 310903/310903, только в эталоне 0, только во втором 0, разные частоты 0
↪  -> совпадает
итог для mapreduce, наивный: ВСЕ ЧИСЛА СОВПАЛИ

--- mapreduce с отказом маппера (results/mapreduce_failtest/) против mapreduce (основной)
отобранные вакансии: 4356 и 4356, набор id совпадает: да
k=1: ключей 1164/1164, сумма частот 22866/22866, только в эталоне 0, только во втором 0, разные частоты 0 ->
↪  совпадает
k=2: ключей 17198/17198, сумма частот 92865/92865, только в эталоне 0, только во втором 0, разные частоты 0 ->
↪  совпадает
k=3: ключей 114332/114332, сумма частот 310903/310903, только в эталоне 0, только во втором 0, разные частоты 0
↪  -> совпадает
итог для mapreduce с отказом маппера: ВСЕ ЧИСЛА СОВПАЛИ

ИТОГ: результаты всех способов совпали полностью
```



```
$ python -m lab.verify
эталон: results/mapreduce/ ; сравниваю с: results/duckdb/, results/mapreduce_naive/, results/mapreduce_failtest/

--- duckdb, csv (results/duckdb/) против mapreduce (основной)
отобранные вакансии: 4356 и 4356, набор id совпадает: да
k=1: ключей 1164/1164, сумма частот 22866/22866, только в эталоне 0, только во втором 0, разные частоты 0 -> совпадает
k=2: ключей 17198/17198, сумма частот 92865/92865, только в эталоне 0, только во втором 0, разные частоты 0 -> совпадает
k=3: ключей 114332/114332, сумма частот 310903/310903, только в эталоне 0, только во втором 0, разные частоты 0 -> совпадает
итог для duckdb, csv: ВСЕ ЧИСЛА СОВПАЛИ

--- mapreduce, наивный (results/mapreduce_naive/) против mapreduce (основной)
отобранные вакансии: 4356 и 4356, набор id совпадает: да
k=1: ключей 1164/1164, сумма частот 22866/22866, только в эталоне 0, только во втором 0, разные частоты 0 -> совпадает
k=2: ключей 17198/17198, сумма частот 92865/92865, только в эталоне 0, только во втором 0, разные частоты 0 -> совпадает
k=3: ключей 114332/114332, сумма частот 310903/310903, только в эталоне 0, только во втором 0, разные частоты 0 -> совпадает
итог для mapreduce, наивный: ВСЕ ЧИСЛА СОВПАЛИ

--- mapreduce с отказом маппера (results/mapreduce_failtest/) против mapreduce (основной)
отобранные вакансии: 4356 и 4356, набор id совпадает: да
k=1: ключей 1164/1164, сумма частот 22866/22866, только в эталоне 0, только во втором 0, разные частоты 0 -> совпадает
k=2: ключей 17198/17198, сумма частот 92865/92865, только в эталоне 0, только во втором 0, разные частоты 0 -> совпадает
k=3: ключей 114332/114332, сумма частот 310903/310903, только в эталоне 0, только во втором 0, разные частоты 0 -> совпадает
итог для mapreduce с отказом маппера: ВСЕ ЧИСЛА СОВПАЛИ

ИТОГ: результаты всех способов совпали полностью

$ python -m lab.growth
вакансий варианта: 4356
сколько навыков в одной вакансии и сколько сочетаний она даёт:
     n  вакансий    доля   C(n,1)   C(n,2)   C(n,3)   вклад в пары / тройки
     0      1011   23.2%        0        0        0    0.0% /   0.0%
     1        72    1.7%        1        0        0    0.0% /   0.0%
     2       163    3.7%        2        1        0    0.2% /   0.0%
     3       341    7.8%        3        3        1    1.1% /   0.1%
     4       448   10.3%        4        6        4    2.9% /   0.6%
     5       492   11.3%        5       10       10    5.3% /   1.6%
     6       366    8.4%        6       15       20    5.9% /   2.4%
     7       309    7.1%        7       21       35    7.0% /   3.5%
     8       240    5.5%        8       28       56    7.2% /   4.3%
     9       231    5.3%        9       36       84    9.0% /   6.2%
    10       209    4.8%       10       45      120   10.1% /   8.1%
    11       117    2.7%       11       55      165    6.9% /   6.2%
    12        94    2.2%       12       66      220    6.7% /   6.7%
    13        45    1.0%       13       78      286    3.8% /   4.1%
    14        47    1.1%       14       91      364    4.6% /   5.5%
    15        32    0.7%       15      105      455    3.6% /   4.7%
    16        42    1.0%       16      120      560    5.4% /   7.6%
    17        21    0.5%       17      136      680    3.1% /   4.6%
    18        16    0.4%       18      153      816    2.6% /   4.2%
    19        28    0.6%       19      171      969    5.2% /   8.7%
    20         7    0.2%       20      190     1140    1.4% /   2.6%
    24         9    0.2%       24      276     2024    2.7% /   5.9%
  ... ещё 14 значений n > 15: 139 вакансий, но 46.1% всех троек

рост по размеру сочетания k:
   k  уникальных  сумма частот  sum C(n,k)  в shuffle (opt)
   1        1164         22866       22866
   2       17198         92865       92865
   3      114332        310903      310903
всего записей в shuffle после combiner: 287150, без combiner: 426634
в среднем 5.25 навыка на вакансию, максимум 30
вакансия с 30 навыками одна даёт 435 пар и 4060 троек
```

**Рис. 7:** Сверка MapReduce, DuckDB, наивной версии и запуска с отказом

## 6.2 Почему пар и троек так много

| $k$ | Уникальных сочетаний | $\sum$ частот | $\sum_i \binom{n_i}{k}$ |
|---|---|---|---|
| 1 | 1 164 | 22 866 | 22 866 |
| 2 | 17 198 | 92 865 | 92 865 |
| 3 | 114 332 | 310 903 | 310 903 |

**Таблица 3:** Рост числа сочетаний с ростом $k$

Число записей растёт так быстро из-за комбинаторики внутри одной вакансии. Вакансия с $n$ навыками даёт $n$ навыков, $\binom{n}{2} = n(n-1)/2$ пар и $\binom{n}{3} = n(n-1)(n-2)/6$ троек. Первое растёт линейно, второе квадратично, третье кубически. В среднем у вакансии 5,25 навыка, а у самой «богатой» их 30: одна такая вакансия даёт 435 пар и 4 060 троек.

Поэтому основную массу троек дают немногие вакансии с длинным списком навыков. Вакансии с 10 и более навыками — это 15,7 % выборки, но на них приходится 81,3 % всех вхождений троек. Число уникальных сочетаний растёт медленнее суммы частот: одинаковые популярные пары повторяются из вакансии в вакансию. Combiner как раз и использует эти повторы.

| $n$ навыков | Вакансий | Доля, % | $\binom{n}{2}$ | $\binom{n}{3}$ | Вклад в пары | Вклад в тройки |
|---|---|---|---|---|---|---|
| 0 | 1 011 | 23,2 | 0 | 0 | 0 | 0 |
| 1 | 72 | 1,7 | 0 | 0 | 0 | 0 |
| 2 | 163 | 3,7 | 1 | 0 | 163 | 0 |
| 3 | 341 | 7,8 | 3 | 1 | 1 023 | 341 |
| 4 | 448 | 10,3 | 6 | 4 | 2 688 | 1 792 |
| 5 | 492 | 11,3 | 10 | 10 | 4 920 | 4 920 |
| 6 | 366 | 8,4 | 15 | 20 | 5 490 | 7 320 |
| 7 | 309 | 7,1 | 21 | 35 | 6 489 | 10 815 |
| 8 | 240 | 5,5 | 28 | 56 | 6 720 | 13 440 |
| 9 | 231 | 5,3 | 36 | 84 | 8 316 | 19 404 |
| 10 | 209 | 4,8 | 45 | 120 | 9 405 | 25 080 |
| 11 | 117 | 2,7 | 55 | 165 | 6 435 | 19 305 |
| 12 | 94 | 2,2 | 66 | 220 | 6 204 | 20 680 |
| $> 12$ | 263 | 6,0 | | | 35 012 | 187 806 |

**Таблица 4:** Распределение числа навыков и вклад в сумму частот

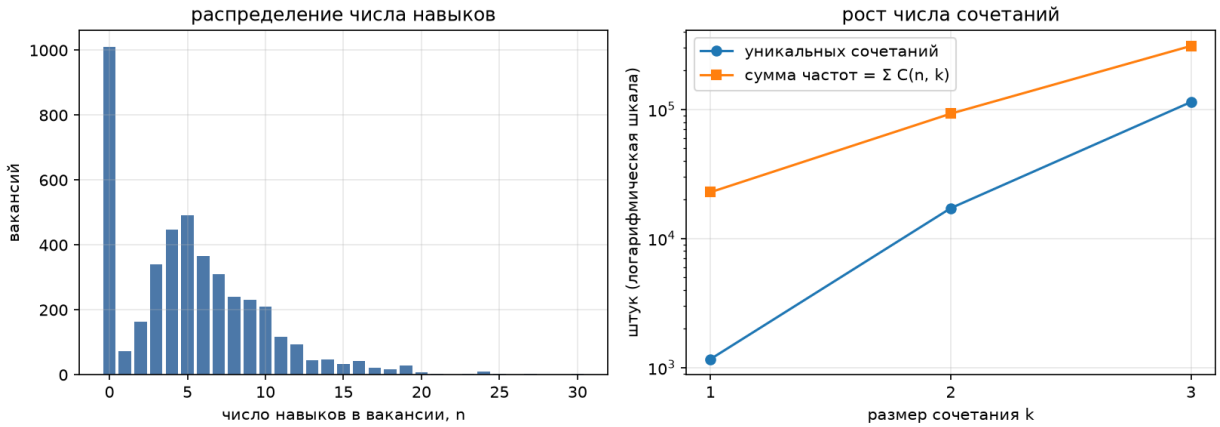



**Рис. 8:** Распределение числа навыков в вакансии и рост числа сочетаний (логарифмическая шкала)

```
$ python -m lab.growth
вакансий варианта: 4356
сколько навыков в одной вакансии и сколько сочетаний она даёт:
    n  вакансий    доля   C(n,1)   C(n,2)   C(n,3)   вклад в пары / тройки
    0      1011   23.2%        0        0        0     0.0% /   0.0%
    1        72    1.7%        1        0        0     0.0% /   0.0%
    2       163    3.7%        2        1        0     0.2% /   0.0%
    3       341    7.8%        3        3        1     1.1% /   0.1%
    4       448   10.3%        4        6        4     2.9% /   0.6%
    5       492   11.3%        5       10       10     5.3% /   1.6%
    6       366    8.4%        6       15       20     5.9% /   2.4%
    7       309    7.1%        7       21       35     7.0% /   3.5%
    8       240    5.5%        8       28       56     7.2% /   4.3%
    9       231    5.3%        9       36       84     9.0% /   6.2%
   10       209    4.8%       10       45      120    10.1% /   8.1%
   11       117    2.7%       11       55      165     6.9% /   6.2%
   12        94    2.2%       12       66      220     6.7% /   6.7%
   13        45    1.0%       13       78      286     3.8% /   4.1%
   14        47    1.1%       14       91      364     4.6% /   5.5%
   15        32    0.7%       15      105      455     3.6% /   4.7%
   16        42    1.0%       16      120      560     5.4% /   7.6%
   17        21    0.5%       17      136      680     3.1% /   4.6%
   18        16    0.4%       18      153      816     2.6% /   4.2%
   19        28    0.6%       19      171      969     5.2% /   8.7%
   20         7    0.2%       20      190     1140     1.4% /   2.6%
   24         9    0.2%       24      276     2024     2.7% /   5.9%
  ... ещё 14 значений n > 15: 139 вакансий, но 46.1% всех троек

рост по размеру сочетания k:
```

```
   k  уникальных  сумма частот  sum C(n,k)  в shuffle (opt)
   1        1164         22866       22866
   2       17198         92865       92865
   3      114332        310903      310903
всего записей в shuffle после combiner: 287150, без combiner: 426634
в среднем 5.25 навыка на вакансию, максимум 30
вакансия с 30 навыками одна даёт 435 пар и 4060 троек
график сохранён: report/plots/growth.png
```

# 7 Этап 5. Навыки и зарплата

## 7.1 Методика

Зарплата известна у 1 442 вакансий варианта (33,1 %). Медиана — 160 000 руб. на руки, межквартильный размах 100 000–250 000 руб.. Я беру медиану, а не среднее, потому что пара вакансий на миллион сдвинула бы среднее куда сильнее, чем реальную картину.

«Прибавку» сочетания я считаю относительно его *базы* — самой частой части, без которой сочетание не существует:

$$\Delta(a,b) = \mathrm{med}(a \wedge b) - \mathrm{med}(A), \qquad A = \arg\max\{\mathrm{freq}(a), \mathrm{freq}(b)\},$$

$$\Delta(a,b,c) = \mathrm{med}(a \wedge b \wedge c) - \mathrm{med}(P), \qquad P = \arg \max_{\text{пары внутри тройки}} \mathrm{freq}.$$

Частоты берутся из результатов MapReduce (k1 и k2). Смысл такой: «сколько добавляет к зарплате специалиста с навыком A ещё и навык B». Базой выбран *более частый* навык, потому что он и есть основа профиля, а редкий навык — добавка к нему. Например, в паре «c# + kubernetes» база — c#, и вопрос звучит так: что даёт Kubernetes C#-разработчику.

Чтобы медиана не прыгала от двух-трёх вакансий, есть пороги: у пары не меньше 30 вакансий с зарплатой, у тройки — не меньше 20, у базы — не меньше 30, плюс не меньше 5 разных работодателей. Иначе «лучшей парой» могла бы оказаться одна компания с десятком одинаковых вакансий. Порог прошли 204 пар и 423 троек.

| Навык | Вакансий | С зарплатой | Медиана, руб. | К медиане варианта |
|---|---|---|---|---|
| c# | 2 237 | 841 | 170 000 | +10 000 |
| .net framework | 1 130 | 432 | 180 000 | +20 000 |
| asp.net | 1 093 | 411 | 174 000 | +14 000 |
| postgresql | 1 031 | 401 | 215 000 | +55 000 |
| sql | 927 | 349 | 170 000 | +10 000 |
| git | 859 | 375 | 170 000 | +10 000 |
| .net core | 771 | 291 | 187 050 | +27 050 |
| ms sql | 671 | 299 | 191 400 | +31 400 |
| c# | 613 | 244 | 185 000 | +25 000 |
| entity framework | 505 | 220 | 180 000 | +20 000 |
| rabbitmq | 450 | 213 | 250 000 | +90 000 |
| ооп | 449 | 180 | 157 500 | −2 500 |
| javascript | 414 | 172 | 150 000 | −10 000 |
| docker | 383 | 134 | 215 000 | +55 000 |
| .net | 331 | 127 | 280 000 | +120 000 |

**Таблица 5:** Самые частые навыки варианта и их медианная зарплата

## 7.2 Пары

В таблицах ниже: «с з/п» — число вакансий сочетания с зарплатой, «база» — более частый навык и его медиана, «джуны» — доля вакансий с опытом до 3 лет у сочетания и у базы.

| Пара | С з/п | База | Мед. базы | Мед. пары | Δ, руб. | Δ, % | Джуны, % |
|---|---|---|---|---|---|---|---|
| .net + ооп | 59 | ооп | 157 500 | 300 000 | +142 500 | 90,5 | 10/51 |
| solid + ооп | 62 | ооп | 157 500 | 300 000 | +142 500 | 90,5 | 13/51 |
| .net + .net framework | 54 | .net framework | 180 000 | 300 000 | +120 000 | 66,7 | 22/37 |
| .net + c# | 41 | c# | 185 000 | 300 000 | +115 000 | 62,2 | 5/37 |
| rabbitmq + c# | 66 | c# | 185 000 | 300 000 | +115 000 | 62,2 | 18/37 |
| solid + c# | 41 | c# | 185 000 | 300 000 | +115 000 | 62,2 | 10/37 |
| ооп + c# | 52 | c# | 185 000 | 300 000 | +115 000 | 62,2 | 27/37 |
| .net core + mongodb | 49 | .net core | 187 050 | 300 000 | +112 950 | 60,4 | 12/30 |
| .net + ms sql | 56 | ms sql | 191 400 | 300 000 | +108 600 | 56,7 | 16/36 |
| ms sql + rabbitmq | 80 | ms sql | 191 400 | 300 000 | +108 600 | 56,7 | 14/36 |

**Таблица 6:** Пары, которые сильнее всего **повышают** медиану относительно базы

| Пара | С з/п | База | Мед. базы | Мед. пары | Δ, руб. | Δ, % | Джуны, % |
|---|---|---|---|---|---|---|---|
| c# + css | 60 | c# | 170 000 | 87 000 | −83 000 | -48,8 | 78/42 |
| git + html | 31 | git | 170 000 | 90 000 | −80 000 | -47,1 | 68/38 |
| sql + visual studio c# | 41 | sql | 170 000 | 100 000 | −70 000 | -41,2 | 71/48 |
| css + html | 53 | html | 150 000 | 80 000 | −70 000 | -46,7 | 81/65 |
| c# + ms visual studio | 64 | c# | 170 000 | 100 000 | −70 000 | -41,2 | 72/42 |
| javascript + postgresql | 39 | postgresql | 215 000 | 150 000 | −65 000 | -30,2 | 44/21 |
| ms sql server + postgresql | 33 | postgresql | 215 000 | 152 250 | −62 750 | -29,2 | 61/21 |
| .net framework + visual studio c# | 32 | .net framework | 180 000 | 120 900 | −59 100 | -32,8 | 62/37 |
| .net framework + ms visual studio | 38 | .net framework | 180 000 | 120 900 | −59 100 | -32,8 | 53/37 |
| ms visual studio + ооп | 36 | ооп | 157 500 | 100 000 | −57 500 | -36,5 | 72/51 |

**Таблица 7:** Пары, которые сильнее всего **понижают** медиану относительно базы

## 7.3 Тройки

| Тройка | С з/п | Базовая пара | Мед. базы | Мед. тройки | Δ, руб. | Δ, % | Джуны, % |
|---|---|---|---|---|---|---|---|
| .net + .net core + c# | 22 | .net core + c# | 177 500 | 320 000 | +142 500 | 80,3 | 18/28 |
| .net + asp.net + c# | 24 | asp.net + c# | 172 000 | 307 500 | +135 500 | 78,8 | 12/38 |
| solid + sql + ооп | 20 | sql + ооп | 146 775 | 280 000 | +133 225 | 90,8 | 25/55 |
| .net core + mongodb + ms sql | 23 | .net core + ms sql | 190 000 | 320 000 | +130 000 | 68,4 | 17/27 |
| c# + kubernetes + sql | 24 | c# + sql | 160 000 | 290 000 | +130 000 | 81,2 | 17/51 |
| asp.net + mongodb + ms sql | 22 | asp.net + ms sql | 172 000 | 300 000 | +128 000 | 74,4 | 0/40 |
| .net core + c# + mongodb | 38 | .net core + c# | 177 500 | 300 000 | +122 500 | 69,0 | 5/28 |
| asp.net + asp.net core + postgresql | 22 | asp.net + postgresql | 185 000 | 300 000 | +115 000 | 62,2 | 5/21 |
| asp.net + mongodb + postgresql | 36 | asp.net + postgresql | 185 000 | 300 000 | +115 000 | 62,2 | 8/21 |
| c# + mongodb + ms sql | 28 | c# + ms sql | 175 000 | 287 500 | +112 500 | 64,3 | 25/42 |

**Таблица 8:** Тройки, которые сильнее всего **повышают** медиану относительно базовой пары

| Тройка | С з/п | Базовая пара | Мед. базы | Мед. тройки | Δ, руб. | Δ, % | Джуны, % |
|---|---|---|---|---|---|---|---|
| .net framework + ms sql + ms visual studio | 20 | .net framework + ms sql | 204 400 | 100 000 | −104 400 | -51,1 | 60/34 |
| .net framework + postgresql + sql | 32 | .net framework + postgresql | 242 450 | 141 375 | −101 075 | -41,7 | 50/23 |
| entity framework + git + javascript | 27 | entity framework + git | 180 000 | 87 000 | −93 000 | -51,7 | 56/43 |
| c# + css + sql | 21 | c# + sql | 160 000 | 70 000 | −90 000 | -56,2 | 86/51 |
| c# + css + git | 24 | c# + git | 170 000 | 83 500 | −86 500 | -50,9 | 88/38 |
| asp.net + c# + ms visual studio | 20 | asp.net + c# | 172 000 | 93 500 | −78 500 | -45,6 | 70/38 |
| c# + html + sql | 26 | c# + sql | 160 000 | 83 500 | −76 500 | -47,8 | 81/51 |
| c# + git + html | 28 | c# + git | 170 000 | 93 500 | −76 500 | -45,0 | 71/38 |
| rabbitmq + solid + sql | 27 | rabbitmq + solid | 300 000 | 225 000 | −75 000 | -25,0 | 11/6 |
| git + rabbitmq + solid | 26 | rabbitmq + solid | 300 000 | 225 000 | −75 000 | -25,0 | 8/6 |

**Таблица 9:** Тройки, которые сильнее всего **понижают** медиану относительно базовой пары

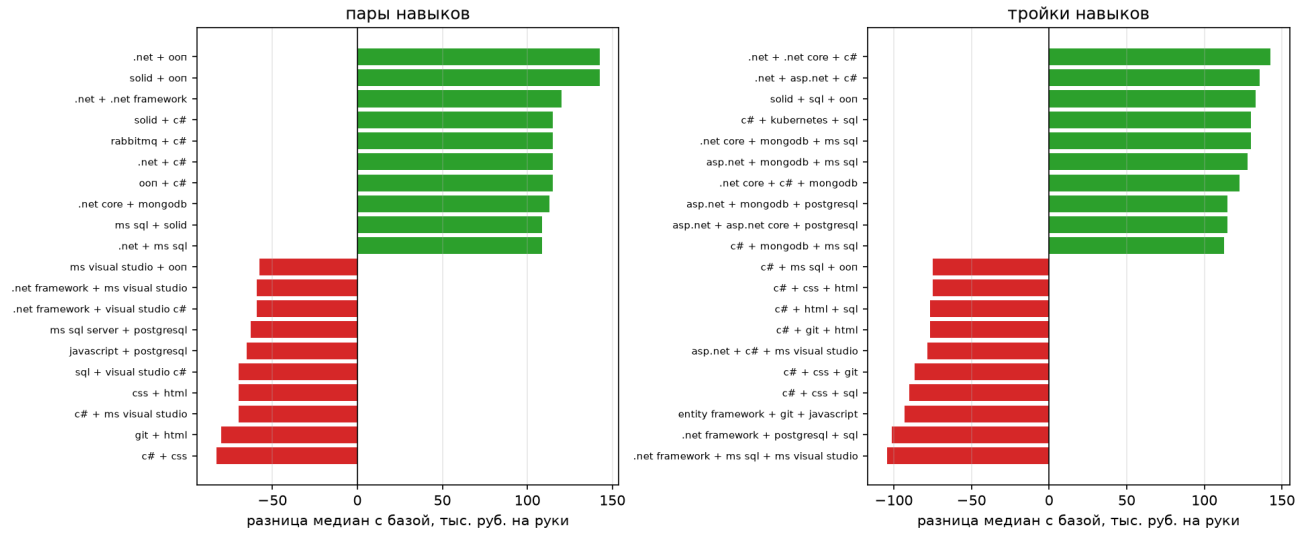



**Рис. 9:** Разница медиан сочетания и его базы: зелёные повышают, красные понижают

## 7.4 Что из этого следует

Сильнее всего медиану поднимает пара **.net + ооп**: на +142 500 руб. к медиане базового навыка ооп. Сильнее всего опускает пара **c# + css**: −83 000 руб. к базе c#. Среди троек лидирует **.net + .net core + c#** (+142 500 руб.), а самая «дешёвая» — **.net framework + ms sql + ms visual studio** (−104 400 руб.).

Важная оговорка: это *связь*, а не причина. Навык сам по себе зарплату не платит, зато он многое говорит о том, *какая это вакансия*. Это хорошо видно по столбцу «джуны». В повышающих сочетаниях вакансий с опытом до трёх лет в среднем 14 %, а в понижающих — 62 %. Доля Москвы и Петербурга — 48 % и 30 % соответственно. Отсюда три типичных механизма:

- **уровень позиции.** Инфраструктурные и архитектурные навыки (Kubernetes, Kafka, микросервисы, highload) обычно пишут в вакансии для middle+ и senior, и медиана растёт вместе с опытом, а не из-за самого слова в описании;
- **тип продукта.** Desktop-стек (WinForms, WPF), связка с 1С, внутренняя автоматизация — это часто небольшие компании и регионы, где ставки ниже, чем в продуктовом вебе;
- **«списки всего подряд».** Вакансии, где работодатель перечисляет и HTML, и SQL, и 1С, нередко ищут универсала на невысокую ставку, поэтому длинный список «базовых» навыков тоже может тянуть медиану вниз.

Отличить один механизм от другого по одной медиане нельзя. Для этого нужна регрессия с опытом, регионом и работодателем как контрольными переменными. Это следующий шаг, за рамками задания.

```
$ python -m lab.salary
вакансий варианта: 4356, из них с зарплатой в рублях: 1442 (33.1%)
зарплата: середина вилки или одна известная граница, только RUR, gross переведён в «на руки» (x0.87), оставлены
↪  значения 10 000..1 000 000 руб.
медиана по варианту: 160 000 руб., квартили 100 000 и 250 000 руб.
прибавка пары = медиана(пара) - медиана(более частый навык пары); тройки = медиана(тройка) - медиана(самая
↪  частая пара внутри)
пороги: пара >= 30 вакансий с з/п, тройка >= 20, база >= 30, работодателей >= 5
прошли порог: пар 204, троек 423

пары, которые сильнее всего ПОВЫШАЮТ медиану относительно базы:
  сочетание                              с з/п база (более частая часть) мед. базы   медиана прибавка      %
  ↪  джун
  .net + ооп                               59 ооп                         157 500   300 000 +142 500  +90.5
  ↪  10%
  solid + ооп                              62 ооп                         157 500   300 000 +142 500  +90.5
  ↪  13%
```

```
  .net + .net framework                    54 .net framework            180 000   300 000 +120 000  +66.7
  ↪  22%
  .net + c#                                41 c#                        185 000   300 000 +115 000  +62.2
  ↪  5%
  rabbitmq + c#                            66 c#                        185 000   300 000 +115 000  +62.2
  ↪  18%
  solid + c#                               41 c#                        185 000   300 000 +115 000  +62.2
  ↪  10%
  ооп + c#                                 52 c#                        185 000   300 000 +115 000  +62.2
  ↪  27%
  .net core + mongodb                      49 .net core                 187 050   300 000 +112 950  +60.4
  ↪  12%
  .net + ms sql                            56 ms sql                    191 400   300 000 +108 600  +56.7
  ↪  16%
  ms sql + rabbitmq                        80 ms sql                    191 400   300 000 +108 600  +56.7
  ↪  14%

пары, которые сильнее всего ПОНИЖАЮТ медиану относительно базы:
  сочетание                          с з/п база (более частая часть) мед. базы   медиана прибавка      %
  ↪  джун
  c# + css                                 60 c#                        170 000    87 000  -83 000  -48.8
  ↪  78%
  git + html                               31 git                       170 000    90 000  -80 000  -47.1
  ↪  68%
  sql + visual studio c#                   41 sql                       170 000   100 000  -70 000  -41.2
  ↪  71%
  css + html                               53 html                      150 000    80 000  -70 000  -46.7
  ↪  81%
  c# + ms visual studio                    64 c#                        170 000   100 000  -70 000  -41.2
  ↪  72%
  javascript + postgresql                  39 postgresql                215 000   150 000  -65 000  -30.2
  ↪  44%
  ms sql server + postgresql               33 postgresql                215 000   152 250  -62 750  -29.2
  ↪  61%
  .net framework + visual studio c#        32 .net framework            180 000   120 900  -59 100  -32.8
  ↪  62%
  .net framework + ms visual studio        38 .net framework            180 000   120 900  -59 100  -32.8
  ↪  53%
  ms visual studio + ооп                   36 ооп                       157 500   100 000  -57 500  -36.5
  ↪  72%

тройки, которые сильнее всего ПОВЫШАЮТ медиану относительно базы:
  сочетание                          с з/п база (более частая часть) мед. базы   медиана прибавка      %
  ↪  джун
  .net + .net core + c#                    22 .net core + c#            177 500   320 000 +142 500  +80.3
  ↪  18%
  .net + asp.net + c#                      24 asp.net + c#              172 000   307 500 +135 500  +78.8
  ↪  12%
  solid + sql + ооп                        20 sql + ооп                 146 775   280 000 +133 225  +90.8
  ↪  25%
  .net core + mongodb + ms sql             23 .net core + ms sql        190 000   320 000 +130 000  +68.4
  ↪  17%
  c# + kubernetes + sql                    24 c# + sql                  160 000   290 000 +130 000  +81.2
  ↪  17%
  asp.net + mongodb + ms sql               22 asp.net + ms sql          172 000   300 000 +128 000  +74.4
  ↪  0%
  .net core + c# + mongodb                 38 .net core + c#            177 500   300 000 +122 500  +69.0
  ↪  5%
  asp.net + asp.net core + postgresql      22 asp.net + postgresql      185 000   300 000 +115 000  +62.2
  ↪  5%
  asp.net + mongodb + postgresql           36 asp.net + postgresql      185 000   300 000 +115 000  +62.2
  ↪  8%
  c# + mongodb + ms sql                    28 c# + ms sql               175 000   287 500 +112 500  +64.3
  ↪  25%

тройки, которые сильнее всего ПОНИЖАЮТ медиану относительно базы:
  сочетание                          с з/п база (более частая часть) мед. базы   медиана прибавка      %
  ↪  джун
  .net framework + ms sql + ms visual st   20 .net framework + ms sql   204 400   100 000 -104 400  -51.1
  ↪  60%
  .net framework + postgresql + sql        32 .net framework + postgre  242 450   141 375 -101 075  -41.7
  ↪  50%
  entity framework + git + javascript      27 entity framework + git    180 000    87 000  -93 000  -51.7
  ↪  56%
  c# + css + sql                           21 c# + sql                  160 000    70 000  -90 000  -56.2
  ↪  86%
  c# + css + git                           24 c# + git                  170 000    83 500  -86 500  -50.9
  ↪  88%
```

```
  asp.net + c# + ms visual studio          20 asp.net + c#              172 000    93 500  -78 500  -45.6
  ↪  70%
  c# + html + sql                          26 c# + sql                  160 000    83 500  -76 500  -47.8
  ↪  81%
  c# + git + html                          28 c# + git                  170 000    93 500  -76 500  -45.0
  ↪  71%
  rabbitmq + solid + sql                   27 rabbitmq + solid          300 000   225 000  -75 000  -25.0
  ↪  11%
  git + rabbitmq + solid                   26 rabbitmq + solid          300 000   225 000  -75 000  -25.0
  ↪  8%

наблюдение: в повышающих сочетаниях вакансий с опытом до 3 лет в среднем 14%, в понижающих 62%
наблюдение: доля москвы и петербурга в повышающих сочетаниях 48%, в понижающих 30%
график сохранён: report/plots/salary.png
все сочетания с прибавками: s3://hh-lake/results/salary/effects.json
```

```
$ python -m lab.salary
вакансий варианта: 4356, из них с зарплатой в рублях: 1442 (33.1%)
зарплата: середина вилки или одна известная граница, только RUR, gross переведён в «на руки» (x0.87), оставлены значения 10
000..1 000 000 руб.
медиана по варианту: 160 000 руб., квартили 100 000 и 250 000 руб.
прибавка пары = медиана(пара) − медиана(более частый навык пары); тройки = медиана(тройка) − медиана(самая частая пара внутр
и)
пороги: пара >= 30 вакансий с з/п, тройка >= 20, база >= 30, работодателей >= 5
прошли порог: пар 204, троек 423

пары, которые сильнее всего ПОВЫШАЮТ медиану относительно базы:
  сочетание                          с з/п база (более частая часть) мед. базы   медиана прибавка      %  джун
  .net + oon                                 59 oon                     157 500   300 000 +142 500  +90.5   10%
  solid + oon                                62 oon                     157 500   300 000 +142 500  +90.5   13%
  .net + .net framework                      54 .net framework          180 000   300 000 +120 000  +66.7   22%
  .net + c#                                  41 c#                      185 000   300 000 +115 000  +62.2    5%
  rabbitmq + c#                              66 c#                      185 000   300 000 +115 000  +62.2   18%
  solid + c#                                 41 c#                      185 000   300 000 +115 000  +62.2   10%
  oon + c#                                   52 c#                      185 000   300 000 +115 000  +62.2   27%
  .net core + mongodb                        49 .net core               187 050   300 000 +112 950  +60.4   12%
  .net + ms sql                              56 ms sql                  191 400   300 000 +108 600  +56.7   16%
  ms sql + rabbitmq                          80 ms sql                  191 400   300 000 +108 600  +56.7   14%

пары, которые сильнее всего ПОНИЖАЮТ медиану относительно базы:
  сочетание                          с з/п база (более частая часть) мед. базы   медиана прибавка      %  джун
  c# + css                                   60 c#                      170 000    87 000  -83 000  -48.8   78%
  git + html                                 31 git                     170 000    90 000  -80 000  -47.1   68%
  sql + visual studio c#                     41 sql                     170 000   100 000  -70 000  -41.2   71%
  css + html                                 53 html                    150 000    80 000  -70 000  -46.7   81%
  c# + ms visual studio                      64 c#                      170 000   100 000  -70 000  -41.2   72%
  javascript + postgresql                    39 postgresql              215 000   150 000  -65 000  -30.2   44%
  ms sql server + postgresql                 33 postgresql              215 000   152 250  -62 750  -29.2   61%
  .net framework + visual studio c#          32 .net framework          180 000   120 900  -59 100  -32.8   62%
  .net framework + ms visual studio          38 .net framework          180 000   120 900  -59 100  -32.8   53%
  ms visual studio + oon                     36 oon                     157 500   100 000  -57 500  -36.5   72%

тройки, которые сильнее всего ПОВЫШАЮТ медиану относительно базы:
  сочетание                          с з/п база (более частая часть) мед. базы   медиана прибавка      %  джун
  .net + .net core + c#                      22 .net core + c#          177 500   320 000 +142 500  +80.3   18%
  .net + asp.net + c#                        24 asp.net + c#            172 000   307 500 +135 500  +78.8   12%
  solid + sql + oon                          20 sql + oon               146 775   280 000 +133 225  +90.8   25%
  .net core + mongodb + ms sql               23 .net core + ms sql      190 000   320 000 +130 000  +68.4   17%
  c# + kubernetes + sql                      24 c# + sql                160 000   290 000 +130 000  +81.2   17%
  asp.net + mongodb + ms sql                 22 asp.net + ms sql        172 000   300 000 +128 000  +74.4    0%
  .net core + c# + mongodb                   38 .net core + c#          177 500   300 000 +122 500  +69.0    5%
  asp.net + asp.net core + postgresql        22 asp.net + postgresql    185 000   300 000 +115 000  +62.2    5%
  asp.net + mongodb + postgresql             36 asp.net + postgresql    185 000   300 000 +115 000  +62.2    8%
  c# + mongodb + ms sql                      28 c# + ms sql             175 000   287 500 +112 500  +64.3   25%

тройки, которые сильнее всего ПОНИЖАЮТ медиану относительно базы:
  сочетание                          с з/п база (более частая часть) мед. базы   медиана прибавка      %  джун
  .net framework + ms sql + ms visual st     20 .net framework + ms sql    204 400   100 000 -104 400  -51.1   60%
  .net framework + postgresql + sql          32 .net framework + postgre   242 450   141 375 -101 075  -41.7   50%
  entity framework + git + javascript        27 entity framework + git     180 000    87 000  -93 000  -51.7   56%
  c# + css + sql                             21 c# + sql                160 000    70 000  -90 000  -56.2   86%
  c# + css + git                             24 c# + git                170 000    83 500  -86 500  -50.9   88%
  asp.net + c# + ms visual studio            20 asp.net + c#            172 000    93 500  -78 500  -45.6   70%
  c# + html + sql                            26 c# + sql                160 000    83 500  -76 500  -47.8   81%
  c# + git + html                            28 c# + git                170 000    93 500  -76 500  -45.0   71%
  rabbitmq + solid + sql                     27 rabbitmq + solid        300 000   225 000  -75 000  -25.0   11%
  git + rabbitmq + solid                     26 rabbitmq + solid        300 000   225 000  -75 000  -25.0    8%

наблюдение: в повышающих сочетаниях вакансий с опытом до 3 лет в среднем 14%, в понижающих 62%
наблюдение: доля москвы и петербурга в повышающих сочетаниях 48%, в понижающих 30%
```

**Рис. 10:** Вывод анализа зарплат

# 8 Этап 6. Три региона

Для сравнения я взял две столицы и самый крупный по числу вакансий регион после них: Москва, Санкт-Петербург, Новосибирск. Третий регион скрипт выбирает автоматически, чтобы в каждом было достаточно данных для медиан.

| Регион | Вакансий | Доля, % | С з/п | Медиана | Q1–Q3 | С навык., % | Навыков | Удал., % | Опыт<3 л., % |
|---|---|---|---|---|---|---|---|---|---|
| Москва | 1 848 | 42,4 | 522 | 216 250 | 161 250–275 000 | 75 | 5,0 | 46 | 26 |
| Санкт-Петербург | 554 | 12,7 | 197 | 180 000 | 120 000–250 000 | 76 | 5,2 | 34 | 32 |
| Новосибирск | 222 | 5,1 | 91 | 158 775 | 107 500–250 000 | 74 | 5,0 | 32 | 46 |

**Таблица 10:** Таблица 1. Профиль регионов

**Таблица 11:** Таблица 2. Самые частые навыки, пары и тройки по регионам (доля — от вакансий региона с навыками, медиана — если в сочетании $\geq 5$ вакансий с зарплатой)

| Регион | $k$ | Сочетание | Вакансий | Доля, % | Медиана, руб. |
|---|---|---|---|---|---|
| Москва | 1 | c# | 915 | 66,4 | 210 000 |
| | 1 | asp.net | 462 | 33,5 | 200 000 |
| | 1 | postgresql | 459 | 33,3 | 250 000 |
| | 1 | .net framework | 442 | 32,1 | 217 500 |
| | 1 | sql | 346 | 25,1 | 225 000 |
| | 1 | git | 341 | 24,7 | 214 075 |
| | 1 | .net core | 335 | 24,3 | 250 000 |
| | 2 | .net framework + c# | 330 | 23,9 | 225 000 |
| | 2 | c# + postgresql | 311 | 22,6 | 250 000 |
| | 2 | asp.net + c# | 282 | 20,5 | 200 000 |
| | 2 | c# + git | 255 | 18,5 | 213 150 |
| | 2 | c# + sql | 238 | 17,3 | 213 150 |
| | 3 | asp.net + c# + postgresql | 122 | 8,9 | 216 900 |
| | 3 | .net framework + c# + git | 105 | 7,6 | 210 000 |
| | 3 | .net framework + c# + postgresql | 105 | 7,6 | 250 000 |
| | 3 | c# + git + postgresql | 102 | 7,4 | 225 000 |
| | 3 | .net framework + c# + ms sql | 100 | 7,3 | 222 500 |
| Санкт-Петербург | 1 | c# | 282 | 67,0 | 180 000 |
| | 1 | .net framework | 143 | 34,0 | 195 750 |
| | 1 | postgresql | 124 | 29,5 | 208 800 |
| | 1 | asp.net | 122 | 29,0 | 174 000 |
| | 1 | sql | 120 | 28,5 | 195 750 |
| | 1 | .net core | 97 | 23,0 | 200 000 |
| | 1 | git | 83 | 19,7 | 160 000 |
| | 2 | .net framework + c# | 96 | 22,8 | 181 350 |
| | 2 | c# + sql | 93 | 22,1 | 197 875 |
| | 2 | asp.net + c# | 83 | 19,7 | 174 000 |
| | 2 | c# + postgresql | 72 | 17,1 | 195 750 |
| | 2 | .net core + c# | 66 | 15,7 | 180 000 |
| | 3 | .net framework + c# + sql | 40 | 9,5 | 182 700 |
| | 3 | c# + postgresql + sql | 37 | 8,8 | 180 000 |
| | 3 | .net framework + asp.net + c# | 32 | 7,6 | 160 000 |
| | 3 | .net framework + c# + postgresql | 31 | 7,4 | 250 000 |
| | 3 | asp.net + c# + git | 29 | 6,9 | 174 000 |
| Новосибирск | 1 | c# | 119 | 72,6 | 200 000 |
| | 1 | .net framework | 73 | 44,5 | 200 000 |
| | 1 | asp.net | 56 | 34,1 | 175 000 |
| | 1 | sql | 40 | 24,4 | 170 000 |
| | 1 | postgresql | 35 | 21,3 | 250 000 |
| | 1 | ms sql | 31 | 18,9 | 275 000 |
| | 1 | c# | 31 | 18,9 | 107 500 |
| | 2 | .net framework + c# | 50 | 30,5 | 210 000 |
| | 2 | asp.net + c# | 48 | 29,3 | 200 000 |
| | 2 | .net framework + asp.net | 29 | 17,7 | 190 000 |
| | 2 | c# + sql | 29 | 17,7 | 225 000 |
| | 2 | c# + postgresql | 26 | 15,9 | 250 000 |
| | 3 | .net framework + asp.net + c# | 24 | 14,6 | 200 000 |
| | 3 | asp.net + c# + sql | 15 | 9,1 | — |
| | 3 | .net framework + angular + c# | 14 | 8,5 | 210 000 |

| Регион | $k$ | Сочетание | Вакансий | Доля, % | Медиана, руб. |
|---|---|---|---|---|---|
| | 3 | asp.net + c# + postgresql | 14 | 8,5 | 180 000 |
| | 3 | .net framework + c# + postgresql | 13 | 7,9 | 250 000 |

| Регион | Навык | Доля в регионе, % | В двух других, % | Во сколько раз чаще |
|---|---|---|---|---|
| Москва | developer express | 1,5 | 0,2 | 8,92 |
| Москва | ef core | 1,6 | 0,3 | 4,67 |
| Москва | алгоритмы | 1,2 | 0,3 | 3,40 |
| Санкт-Петербург | разработка логистики | 5,2 | 2,1 | 2,44 |
| Санкт-Петербург | it | 5,2 | 2,8 | 1,87 |
| Санкт-Петербург | linux | 8,3 | 4,9 | 1,69 |
| Новосибирск | английский язык | 9,1 | 2,7 | 3,43 |
| Новосибирск | angular | 11,6 | 4,0 | 2,89 |
| Новосибирск | ооп | 16,5 | 9,9 | 1,66 |

**Таблица 12:** Навыки, которые отличают регион от двух других (не меньше 15 упоминаний)

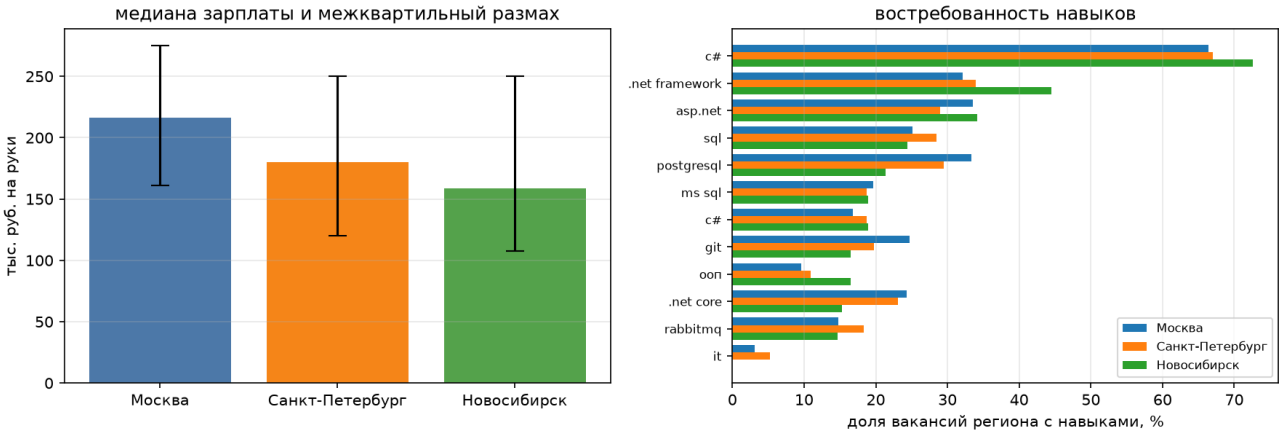



**Рис. 11:** Медианная зарплата с межквартильным размахом и востребованность самых частых навыков по регионам

## 8.1 Наблюдения

- **Зарплата.** Выше всех медиана в регионе «Москва» (216 250 руб.), ниже всех — в «Новосибирск» (158 775 руб.). Разрыв около 36 %. Межквартильный размах показывает, насколько разнородны вакансии внутри региона: чем он шире, тем больше в регионе одновременно и стажёрских позиций, и дорогих ведущих.
- **Опыт.** Доля вакансий с опытом до трёх лет: Москва — 26 %, Санкт-Петербург — 32 %, Новосибирск — 46 %. Если в регионе с низкой медианой больше младших позиций, часть разрыва в зарплате объясняется составом вакансий, а не только региональными ставками.
- **Удалёнка.** Больше всего удалённых вакансий в регионе «Москва» (46 %). Удалённые вакансии со столичной вилкой формально приписаны к региону работодателя, и это сглаживает межрегиональные различия.
- **Стек.** Верхушка рейтинга навыков во всех трёх регионах похожа (таблица 2), а отличия видны во «втором эшелоне»: для региона Москва характернее всего developer express, для Санкт-Петербург — разработка логистики, для Новосибирск — английский язык (см. таблицу выше).

```
$ python -m lab.regions
топ-8 регионов по числу вакансий варианта:
```

```
  Москва                          1848  (42.4%)
  Санкт-Петербург                  554  (12.7%)
  Новосибирск                      222  (5.1%)
  Екатеринбург                     191  (4.4%)
  Казань                           160  (3.7%)
  Нижний Новгород                  113  (2.6%)
  Ростов-на-Дону                    81  (1.9%)
  Самара                            69  (1.6%)
для сравнения беру: Москва, Санкт-Петербург, Новосибирск (две столицы и самый крупный регион после них)

таблица 1. профиль регионов
  регион             вакансий   доля  с з/п   медиана        q1        q3 с навыками  навыков удалёнка  опыт<3л
  ↪  работод.
  Москва                 1848  42.4%    522   216 250   161 250   275 000      74.6%     5.03    45.7%    26.1%
  ↪  709
  Санкт-Петербург         554  12.7%    197   180 000   120 000   250 000      76.0%     5.23    33.9%    31.9%
  ↪  251
  Новосибирск             222   5.1%     91   158 775   107 500   250 000      73.9%     4.96    32.0%    45.9%
  ↪  95

таблица 2. самые частые навыки, пары и тройки в каждом регионе
  (доля - от вакансий региона с навыками; медиана - по вакансиям сочетания с зарплатой, если их >= 5)
  Москва:
    k=1 #1: c#                                                    915  66.4%  медиана 210 000 (n=317)
    k=1 #2: asp.net                                               462  33.5%  медиана 200 000 (n=151)
    k=1 #3: postgresql                                            459  33.3%  медиана 250 000 (n=164)
    k=1 #4: .net framework                                        442  32.1%  медиана 217 500 (n=139)
    k=1 #5: sql                                                   346  25.1%  медиана 225 000 (n=107)
    k=1 #6: git                                                   341  24.7%  медиана 214 075 (n=154)
    k=1 #7: .net core                                             335  24.3%  медиана 250 000 (n=125)
    k=2 #1: .net framework + c#                                   330  23.9%  медиана 225 000 (n=109)
    k=2 #2: c# + postgresql                                       311  22.6%  медиана 250 000 (n=123)
    k=2 #3: asp.net + c#                                          282  20.5%  медиана 200 000 (n=104)
    k=2 #4: c# + git                                              255  18.5%  медиана 213 150 (n=118)
    k=2 #5: c# + sql                                              238  17.3%  медиана 213 150 (n=79)
    k=3 #1: asp.net + c# + postgresql                             122   8.9%  медиана 216 900 (n=50)
    k=3 #2: .net framework + c# + git                             105   7.6%  медиана 210 000 (n=50)
    k=3 #3: .net framework + c# + postgresql                      105   7.6%  медиана 250 000 (n=46)
    k=3 #4: c# + git + postgresql                                 102   7.4%  медиана 225 000 (n=58)
    k=3 #5: .net framework + c# + ms sql                          100   7.3%  медиана 222 500 (n=40)
  Санкт-Петербург:
    k=1 #1: c#                                                    282  67.0%  медиана 180 000 (n=120)
    k=1 #2: .net framework                                        143  34.0%  медиана 195 750 (n=57)
    k=1 #3: postgresql                                            124  29.5%  медиана 208 800 (n=50)
    k=1 #4: asp.net                                               122  29.0%  медиана 174 000 (n=51)
    k=1 #5: sql                                                   120  28.5%  медиана 195 750 (n=52)
    k=1 #6: .net core                                              97  23.0%  медиана 200 000 (n=34)
    k=1 #7: git                                                    83  19.7%  медиана 160 000 (n=41)
    k=2 #1: .net framework + c#                                    96  22.8%  медиана 181 350 (n=40)
    k=2 #2: c# + sql                                               93  22.1%  медиана 197 875 (n=38)
    k=2 #3: asp.net + c#                                           83  19.7%  медиана 174 000 (n=37)
    k=2 #4: c# + postgresql                                        72  17.1%  медиана 195 750 (n=27)
    k=2 #5: .net core + c#                                         66  15.7%  медиана 180 000 (n=21)
    k=3 #1: .net framework + c# + sql                              40   9.5%  медиана 182 700 (n=15)
    k=3 #2: c# + postgresql + sql                                  37   8.8%  медиана 180 000 (n=13)
    k=3 #3: .net framework + asp.net + c#                          32   7.6%  медиана 160 000 (n=13)
    k=3 #4: .net framework + c# + postgresql                       31   7.4%  медиана 250 000 (n=12)
    k=3 #5: asp.net + c# + git                                     29   6.9%  медиана 174 000 (n=12)
  Новосибирск:
    k=1 #1: c#                                                    119  72.6%  медиана 200 000 (n=42)
    k=1 #2: .net framework                                         73  44.5%  медиана 200 000 (n=34)
    k=1 #3: asp.net                                                56  34.1%  медиана 175 000 (n=27)
    k=1 #4: sql                                                    40  24.4%  медиана 170 000 (n=13)
    k=1 #5: postgresql                                             35  21.3%  медиана 250 000 (n=19)
    k=1 #6: ms sql                                                 31  18.9%  медиана 275 000 (n=16)
    k=1 #7: c#                                                     31  18.9%  медиана 107 500 (n=19)
    k=2 #1: .net framework + c#                                    50  30.5%  медиана 210 000 (n=22)
    k=2 #2: asp.net + c#                                           48  29.3%  медиана 200 000 (n=19)
    k=2 #3: .net framework + asp.net                               29  17.7%  медиана 190 000 (n=18)
    k=2 #4: c# + sql                                               29  17.7%  медиана 225 000 (n=7)
    k=2 #5: c# + postgresql                                        26  15.9%  медиана 250 000 (n=13)
    k=3 #1: .net framework + asp.net + c#                          24  14.6%  медиана 200 000 (n=13)
    k=3 #2: asp.net + c# + sql                                     15   9.1%  медиана --- (n=2)
    k=3 #3: .net framework + angular + c#                          14   8.5%  медиана 210 000 (n=10)
    k=3 #4: asp.net + c# + postgresql                              14   8.5%  медиана 180 000 (n=9)
    k=3 #5: .net framework + c# + postgresql                       13   7.9%  медиана 250 000 (n=8)

что отличает регионы: навыки, доля которых в регионе заметно выше, чем в двух других вместе
```

```
  Москва: developer express 1.5% против 0.2% (x8.92); ef core 1.6% против 0.3% (x4.67); алгоритмы 1.2% против
  ↪  0.3% (x3.40)
  Санкт-Петербург: разработка логистики 5.2% против 2.1% (x2.44); it 5.2% против 2.8% (x1.87); linux 8.3% против
  ↪  4.9% (x1.69)
  Новосибирск: английский язык 9.1% против 2.7% (x3.43); angular 11.6% против 4.0% (x2.89); ооп 16.5% против
  ↪  9.9% (x1.66)

таблицы сохранены: s3://hh-lake/results/regions/profile.csv и top_combos.csv
график сохранён: report/plots/regions.png
```

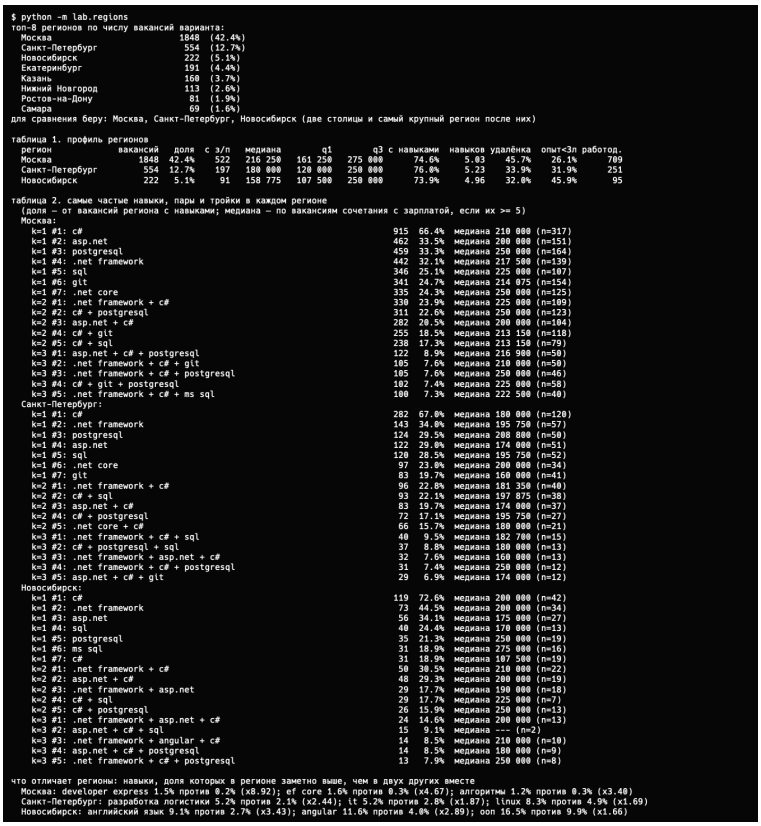



**Рис. 12:** Вывод регионального анализа

# 9 Профилирование и бонус

## 9.1 Как мерил

Каждый способ запускается отдельным процессом 3 раза. Порядок способов в каждой попытке сдвигается, чтобы ни один не оказывался всегда первым или последним. Перед каждым запуском контейнер RustFS перезапускается, поэтому кэши хранилища у всех одинаково «холодные».

- **время** — настенное, от старта процесса до записи результатов на диск, включая запуск интерпретатора и чтение из S3;
- **CPU** — сумма пользовательского и системного времени всех дочерних процессов (`getrusage`). Если CPU больше настенного времени, значит, работало несколько ядер;
- **пик памяти** — максимум суммарного RSS всего дерева процессов (мастер плюс все мапперы), опрос каждые 20 мс через `psutil`. Общие страницы разделяемых библиотек при этом считаются в каждом процессе, так что для многопроцессных вариантов это оценка сверху.

После каждого запуска результат сверяется с эталоном по SHA-256, итог: все совпали — да.

### 9.2 Бонус: что оптимизировал

1. **MapReduce «до»**: один процесс, стандартный модуль `csv`, без combiner'а. В shuffle уходит каждая пара (ключ, 1), редьюсер сортирует и суммирует.
2. **MapReduce «после»**:
   - CSV разбирается C++-парсером pyarrow, и только по нужным колонкам. Длинное описание вакансии в Python-строки вообще не превращается;
   - вакансии отсеиваются по роли ещё до создания словаря строки;
   - combiner в маппере;
   - несколько мапперов параллельно.

   Отдельно замерены промежуточные шаги (только парсер и combiner на одном маппере) и размер чанка 8 и 128 МиБ.
3. **DuckDB «до»** — запрос по исходному CSV. **«После»** — по колоночной копии в Parquet (zstd), которую DuckDB сам записал в тот же бакет. В неё вошли только поля, нужные анализу: vacancy_id, published_at, area, employer_id, employer_name, name, role, salary_from, salary_to, currency, gross, experience, schedule, key_skills. Размер 8.02 миб против 808.35 миб у CSV, в 100,8 раза меньше.

```
==== бонус: колоночная копия датасета в parquet (duckdb пишет её прямо в s3) ====

$ python -m lab.duck --to-parquet
в parquet беру колонки: vacancy_id, published_at, area, employer_id, employer_name, name, role, salary_from,
↪  salary_to, currency, gross, experience, schedule, key_skills
отбрасываю (анализу не нужны): source, status, snapshot_date, employer_inn, employer_ogrn, sphere, employment,
↪  description, url
csv -> parquet через duckdb (httpfs) за 2.5 с
s3://hh-lake/raw/it_vacancies_2024.csv: 847619384 байт (808.35 миб)
s3://hh-lake/raw/it_vacancies_2024.parquet: 8412829 байт (8.02 миб), в 100.8 раза меньше

$ python -m lab.duck --source parquet --tag duckdb_parquet
доступ к s3 из duckdb: httpfs, источник: parquet, потоков 4, лимит памяти 1500MB
вакансий варианта: 4356 (с навыками 3345)
k=1: уникальных сочетаний 1164, сумма частот 22866; топ: c# = 2237
k=2: уникальных сочетаний 17198, сумма частот 92865; топ: .net framework + c# = 793
k=3: уникальных сочетаний 114332, сумма частот 310903; топ: asp.net + c# + postgresql = 266
время: всего 0.59 с, из них чтение и фильтр 0.31 с
  сохранено s3://hh-lake/results/duckdb_parquet/k1.csv (23779 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/duckdb_parquet/k2.csv (482273 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/duckdb_parquet/k3.csv (4142254 байт), sha256 после чтения обратно совпал: да
  сохранено s3://hh-lake/results/duckdb_parquet/vacancy_ids.txt (41014 байт), sha256 после чтения обратно
  ↪  совпал: да
  сохранено s3://hh-lake/results/duckdb_parquet/summary.json (346 байт), sha256 после чтения обратно совпал: да

$ python -m lab.verify
эталон: results/mapreduce/ ; сравниваю с: results/duckdb/, results/mapreduce_naive/,
↪  results/mapreduce_failtest/, results/duckdb_parquet/

--- duckdb, csv (results/duckdb/) против mapreduce (основной)
отобранные вакансии: 4356 и 4356, набор id совпадает: да
k=1: ключей 1164/1164, сумма частот 22866/22866, только в эталоне 0, только во втором 0, разные частоты 0 ->
↪  совпадает
k=2: ключей 17198/17198, сумма частот 92865/92865, только в эталоне 0, только во втором 0, разные частоты 0 ->
↪  совпадает
k=3: ключей 114332/114332, сумма частот 310903/310903, только в эталоне 0, только во втором 0, разные частоты 0
↪  -> совпадает
итог для duckdb, csv: ВСЕ ЧИСЛА СОВПАЛИ

--- mapreduce, наивный (results/mapreduce_naive/) против mapreduce (основной)
отобранные вакансии: 4356 и 4356, набор id совпадает: да
k=1: ключей 1164/1164, сумма частот 22866/22866, только в эталоне 0, только во втором 0, разные частоты 0 ->
↪  совпадает
k=2: ключей 17198/17198, сумма частот 92865/92865, только в эталоне 0, только во втором 0, разные частоты 0 ->
↪  совпадает
k=3: ключей 114332/114332, сумма частот 310903/310903, только в эталоне 0, только во втором 0, разные частоты 0
↪  -> совпадает
итог для mapreduce, наивный: ВСЕ ЧИСЛА СОВПАЛИ

--- mapreduce с отказом маппера (results/mapreduce_failtest/) против mapreduce (основной)
отобранные вакансии: 4356 и 4356, набор id совпадает: да
```

```
k=1: ключей 1164/1164, сумма частот 22866/22866, только в эталоне 0, только во втором 0, разные частоты 0 ->
↪  совпадает
k=2: ключей 17198/17198, сумма частот 92865/92865, только в эталоне 0, только во втором 0, разные частоты 0 ->
↪  совпадает
k=3: ключей 114332/114332, сумма частот 310903/310903, только в эталоне 0, только во втором 0, разные частоты 0
↪  -> совпадает
итог для mapreduce с отказом маппера: ВСЕ ЧИСЛА СОВПАЛИ

--- duckdb, parquet (results/duckdb_parquet/) против mapreduce (основной)
отобранные вакансии: 4356 и 4356, набор id совпадает: да
k=1: ключей 1164/1164, сумма частот 22866/22866, только в эталоне 0, только во втором 0, разные частоты 0 ->
↪  совпадает
k=2: ключей 17198/17198, сумма частот 92865/92865, только в эталоне 0, только во втором 0, разные частоты 0 ->
↪  совпадает
k=3: ключей 114332/114332, сумма частот 310903/310903, только в эталоне 0, только во втором 0, разные частоты 0
↪  -> совпадает
итог для duckdb, parquet: ВСЕ ЧИСЛА СОВПАЛИ

ИТОГ: результаты всех способов совпали полностью
```

## 9.3 Результаты

| Способ | Запусков | Время, с | Мин., с | CPU, с | Пик памяти, МиБ | Совпало |
|---|---|---|---|---|---|---|
| mapreduce, наивный (до) | 3 | 14,71 ± 0,34 | 14,37 | 17,3 | 523 ± 19 | да |
| mapreduce, arrow + combiner, 1 маппер | 3 | 7,44 ± 0,11 | 7,38 | 11,0 | 583 ± 1 | да |
| mapreduce, оптимизированный (после) | 3 | 6,23 ± 0,11 | 6,12 | 18,2 | 1133 ± 12 | да |
| mapreduce opt, чанк 8 миб | 3 | 7,03 ± 0,04 | 7,00 | 17,2 | 779 ± 14 | да |
| mapreduce opt, чанк 128 миб | 3 | 6,26 ± 0,05 | 6,20 | 18,2 | 2052 ± 6 | да |
| duckdb по csv | 3 | 4,45 ± 0,01 | 4,45 | 3,7 | 2529 ± 4 | да |
| duckdb по parquet (после) | 3 | 2,03 ± 0,07 | 1,98 | 2,3 | 253 ± 10 | да |

**Таблица 13:** Время и память (среднее ± стандартное отклонение)

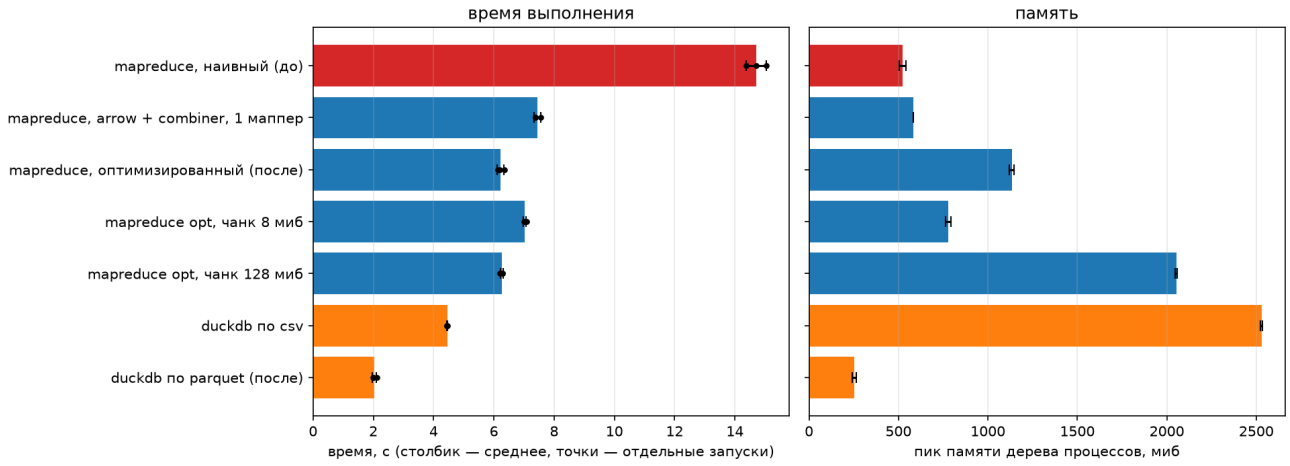



**Рис. 13:** Время и пик памяти каждого способа; точки — отдельные запуски

- **MapReduce: до и после.** Оптимизированная версия быстрее наивной в 2,4 раза (14,71 с против 6,23 с). Основной выигрыш даёт парсер: только замена `csv` на pyarrow с combiner'ом на одном маппере даёт ускорение в 2,0 раза. Переход с одного маппера на несколько меняет время в 1,2 раза. Выигрыш здесь ограничен, потому что мастер читает S3 последовательно, а разбор у pyarrow и так быстрый, и узким местом становится чтение.
- **Память MapReduce.** Пик у наивной версии 523 МиБ, у оптимизированной 1133 МиБ. Combiner сжимает shuffle, но несколько мапперов одновременно держат свои чанки, а RSS каждого процесса считается отдельно. Размер чанка напрямую управляет памятью: при 8 МиБ пик 779 МиБ, при 128 МиБ — 2052 МиБ, а время почти не меняется. Для ноутбука разумный выбор — 16–32 МиБ.

- **DuckDB против своего MapReduce.** DuckDB по CSV (4,45 с) работает в 1,4 раза быстрее моей оптимизированной версии. У него векторизованный многопоточный разбор CSV, фильтр проверяется до материализации строк, а сочетания считаются хэш-соединениями внутри одного процесса, без сериализации данных между процессами. Платим памятью: 2529 МиБ, потому что буферы чтения и хэш-таблицы самосоединения живут в одном процессе.
- **Parquet.** DuckDB по колоночной копии быстрее, чем по CSV, в 2,2 раза и требует в 10,0 раза меньше памяти (253 МиБ). Из S3 читаются только нужные колонки нужных групп строк, а разбор текста не нужен вообще. Это главный практический вывод: если по данным считают больше одного раза, их стоит один раз переложить в колоночный формат.

**Таблица 14:** Все запуски

| Способ | Попытка | Время, с | CPU, с | Пик памяти, МиБ |
|---|---|---|---|---|
| mapreduce, наивный (до) | 1 | 14,37 | 17,0 | 509 |
| mapreduce, наивный (до) | 2 | 14,71 | 17,4 | 545 |
| mapreduce, наивный (до) | 3 | 15,04 | 17,7 | 516 |
| mapreduce, arrow + combiner, 1 маппер | 1 | 7,38 | 10,9 | 583 |
| mapreduce, arrow + combiner, 1 маппер | 2 | 7,38 | 11,0 | 582 |
| mapreduce, arrow + combiner, 1 маппер | 3 | 7,56 | 11,3 | 583 |
| mapreduce, оптимизированный (после) | 1 | 6,20 | 18,0 | 1144 |
| mapreduce, оптимизированный (после) | 2 | 6,35 | 18,5 | 1121 |
| mapreduce, оптимизированный (после) | 3 | 6,12 | 17,9 | 1135 |
| mapreduce opt, чанк 8 миб | 1 | 7,08 | 17,3 | 789 |
| mapreduce opt, чанк 8 миб | 2 | 7,00 | 17,3 | 785 |
| mapreduce opt, чанк 8 миб | 3 | 7,01 | 16,9 | 763 |
| mapreduce opt, чанк 128 миб | 1 | 6,20 | 18,1 | 2046 |
| mapreduce opt, чанк 128 миб | 2 | 6,28 | 18,2 | 2056 |
| mapreduce opt, чанк 128 миб | 3 | 6,28 | 18,3 | 2056 |
| duckdb по csv | 1 | 4,45 | 3,7 | 2527 |
| duckdb по csv | 2 | 4,46 | 3,7 | 2526 |
| duckdb по csv | 3 | 4,45 | 3,7 | 2534 |
| duckdb по parquet (после) | 1 | 2,00 | 2,4 | 242 |
| duckdb по parquet (после) | 2 | 1,98 | 2,2 | 260 |
| duckdb по parquet (после) | 3 | 2,11 | 2,3 | 258 |

```
==== профилирование: каждый способ запускается 3 раза, перед каждым запуском rustfs перезапускается ====
время - от старта процесса до записи результатов; память - пик суммарного rss дерева процессов (опрос каждые 20
↪  мс)
после каждого запуска результат сверяется с эталоном results/mapreduce по sha256

--- попытка 1 из 3
mapreduce, наивный (до)               #1: время  14.37 с, cpu  16.95 с, пик    509 миб, совпало с эталоном: да
mapreduce, arrow + combiner, 1 маппер  #1: время   7.38 с, cpu  10.93 с, пик    583 миб, совпало с эталоном: да
mapreduce, оптимизированный (после)    #1: время   6.20 с, cpu  18.04 с, пик   1144 миб, совпало с эталоном: да
mapreduce opt, чанк 8 миб              #1: время   7.08 с, cpu  17.28 с, пик    789 миб, совпало с эталоном: да
mapreduce opt, чанк 128 миб            #1: время   6.20 с, cpu  18.08 с, пик   2045 миб, совпало с эталоном: да
duckdb по csv                          #1: время   4.45 с, cpu   3.65 с, пик   2527 миб, совпало с эталоном: да
duckdb по parquet (после)              #1: время   2.00 с, cpu   2.42 с, пик    242 миб, совпало с эталоном: да

--- попытка 2 из 3
mapreduce, arrow + combiner, 1 маппер  #2: время   7.38 с, cpu  10.97 с, пик    582 миб, совпало с эталоном: да
mapreduce, оптимизированный (после)    #2: время   6.35 с, cpu  18.47 с, пик   1121 миб, совпало с эталоном: да
mapreduce opt, чанк 8 миб              #2: время   7.00 с, cpu  17.34 с, пик    785 миб, совпало с эталоном: да
mapreduce opt, чанк 128 миб            #2: время   6.28 с, cpu  18.19 с, пик   2056 миб, совпало с эталоном: да
duckdb по csv                          #2: время   4.46 с, cpu   3.72 с, пик   2526 миб, совпало с эталоном: да
duckdb по parquet (после)              #2: время   1.98 с, cpu   2.16 с, пик    260 миб, совпало с эталоном: да
mapreduce, наивный (до)                #2: время  14.71 с, cpu  17.38 с, пик    545 миб, совпало с эталоном: да

--- попытка 3 из 3
mapreduce, оптимизированный (после)    #3: время   6.12 с, cpu  17.94 с, пик   1135 миб, совпало с эталоном: да
```

```
mapreduce opt, чанк 8 миб              #3: время   7.02 с, cpu  16.90 с, пик    763 миб, совпало с эталоном: да
mapreduce opt, чанк 128 миб            #3: время   6.28 с, cpu  18.25 с, пик   2056 миб, совпало с эталоном: да
duckdb по csv                          #3: время   4.44 с, cpu   3.68 с, пик   2534 миб, совпало с эталоном: да
duckdb по parquet (после)              #3: время   2.11 с, cpu   2.29 с, пик    257 миб, совпало с эталоном: да
mapreduce, наивный (до)                #3: время  15.04 с, cpu  17.69 с, пик    516 миб, совпало с эталоном: да
mapreduce, arrow + combiner, 1 маппер  #3: время   7.56 с, cpu  11.25 с, пик    583 миб, совпало с эталоном: да

==== итог профилирования ====
способ                                 n время, с (ср +- ск)    мин  cpu, с      пик, миб совпало
mapreduce, наивный (до)                3       14.71 +- 0.34    14.37   17.34    523 +- 19        да
mapreduce, arrow + combiner, 1 маппер  3        7.44 +- 0.11     7.38   11.05    583 +- 1         да
mapreduce, оптимизированный (после)    3        6.23 +- 0.11     6.12   18.15   1133 +- 12        да
mapreduce opt, чанк 8 миб              3        7.03 +- 0.04     7.00   17.17    779 +- 14        да
mapreduce opt, чанк 128 миб            3        6.26 +- 0.05     6.20   18.17   2052 +- 6         да
duckdb по csv                          3        4.45 +- 0.01     4.45    3.68   2529 +- 4         да
duckdb по parquet (после)              3        2.03 +- 0.07     1.98    2.29    253 +- 10        да

все запуски: s3://hh-lake/results/profile/runs.csv
график сохранён: report/plots/profile.png
```

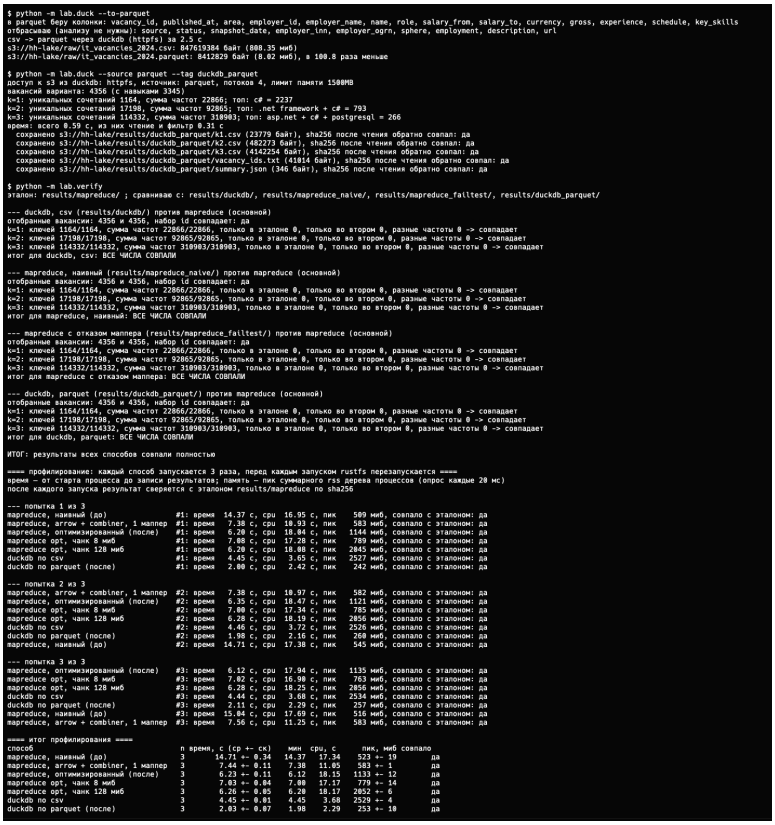



**Рис. 14:** Итог профилирования

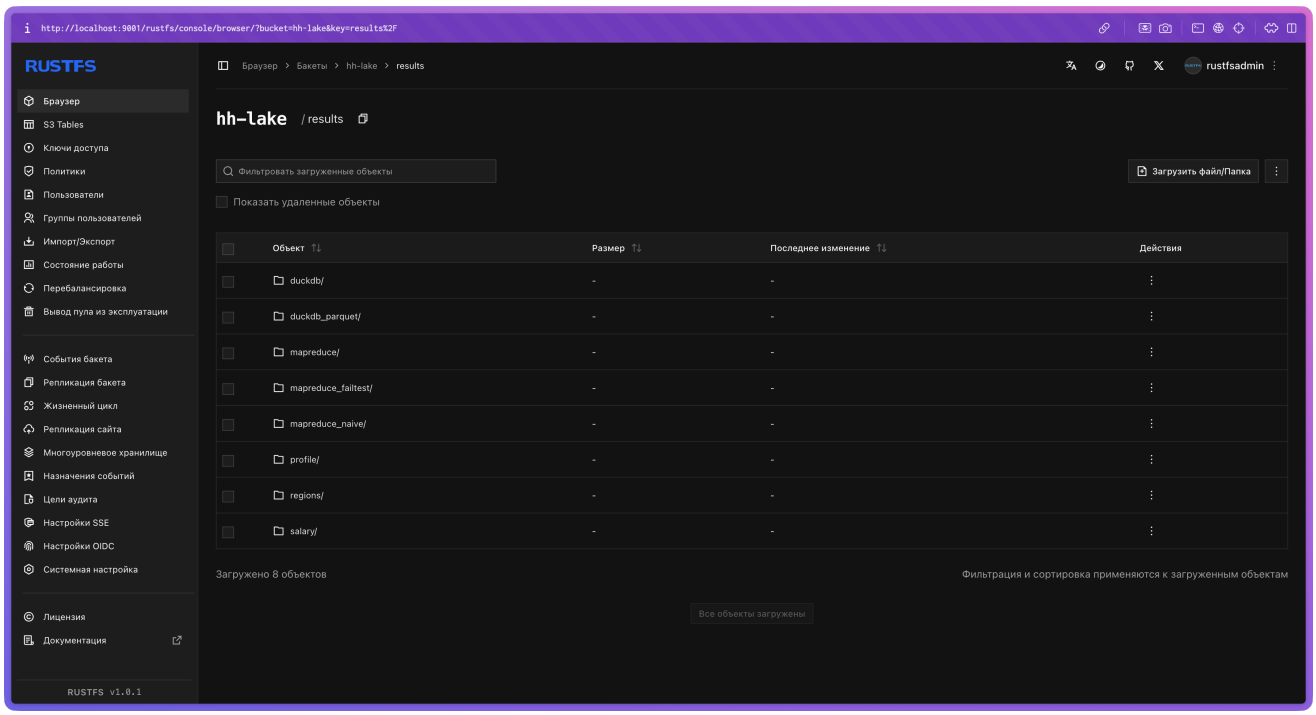



**Рис. 15:** Результаты k1, k2, k3 в бакете hh-lake (results/mapreduce)

# 10 Выводы

1. Хранилище RustFS поднято на четырёх папках-дисках, датасет лежит в бакете `hh-lake`, и после загрузки его SHA-256 совпал с исходным. Все вычисления читают данные только из S3.
2. Мой MapReduce читает объект чанками через Range GET и правильно режет CSV по границам записей, даже когда внутри полей есть переводы строк. Он считает навыки, пары и тройки для 4 356 вакансий варианта C#/.NET и переживает падение процесса-маппера без изменения результата.
3. DuckDB с независимо написанным SQL-условием дал те же 1 164 навыков, 17 198 пар и 114 332 троек с теми же частотами. Совпали все ключи, все частоты и набор отобранных вакансий.
4. Сумма частот растёт от 22 866 к 92 865 и 310 903, потому что вакансия с $n$ навыками даёт $\binom{n}{2}$ пар и $\binom{n}{3}$ троек. Основной вклад дают вакансии с длинными списками навыков.
5. Сильнее всего медиану повышают .net + ооп (пара) и .net + .net core + c# (тройка), понижают — c# + css и .net framework + ms sql + ms visual studio. Во многом это отражает уровень позиции и тип продукта, а не «цену» навыка.
6. Из трёх регионов (Москва, Санкт-Петербург, Новосибирск) выше всего медиана в регионе «Москва», ниже всего — в «Новосибирск», разрыв около 36 %.
7. DuckDB быстрее моего MapReduce, а переход на Parquet ускоряет его ещё в 2,2 раза и экономит память в 10,0 раза. Свой MapReduce после оптимизации стал быстрее в 2,4 раза.

# A Репозиторий и воспроизведение

Исходный код всех программ и отчёт опубликованы на GitHub:

`github.com/seichikk/hh-mapreduce`

```
bash scripts/00_check.sh        # стенд, сборка образа, проверка датасета
bash scripts/01_storage.sh      # rustfs, бакет hh-lake, загрузка, sha256
bash scripts/02_mapreduce.sh    # свой mapreduce (оптимизированный и наивный)
bash scripts/03_failure.sh      # отказ маппера
```

```
bash scripts/04_duckdb.sh       # duckdb
bash scripts/05_verify.sh       # сверка и рост числа сочетаний
bash scripts/06_salary.sh       # зарплаты
bash scripts/07_regions.sh      # регионы
bash scripts/08_profile.sh      # parquet и профилирование
bash scripts/09_report.sh       # этот отчёт
```